



CENTRUL UNIVERSITAR NORD BAIA MARE
FACULTATEA DE INGINERIE
DEPARTAMENTUL DE INGINERIE ELECTRICĂ, ELECTRONICĂ ȘI
CALCULATOARE
Programul de studii: CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

SISTEM DE RECOMANDARE BAZAT PE CONȚINUT

LUCRARE DE LICENȚĂ

Absolvent : **Vancea Radu Bogdan**

Coordonator științific: **Prof. Dr. Habil. Ing. Oliviu Matei**
Drd. Ing. Erdei Rudolf

Iulie, 2024

Cuprins

Capitolul 1	Introducere	1
1.1	Contextul proiectului	1
1.2	Importanța sistemelor de recomandare	1
Capitolul 2	Scopul și obiectivele proiectului	5
2.1	Scopul proiectului	5
2.2	Obiectivele proiectului	6
2.3	Descrierea cerințelor	6
Capitolul 3	Stadiul actual al cercetării	8
3.1	Algoritm de recomandare bazat pe ontologii	8
3.2	Sistem bazat pe coeficientul de atenție	8
3.3	Paradigme Contextuale în Sistemele de Recomandare: Prefiltrare, Postfiltrare și Modelare	9
3.4	Optimizarea filtrării colaborative: LRA vs DNN	9
3.5	Sistemul de recomandare Netflix	9
3.6	Rețea bayesiană pentru recomandări personalizate	10
3.7	Sistem de recomandare hibrid	11
3.8	Folosirea algoritmilor de clustering pentru popularea interacțiunilor	12
3.9	Metode liniare pentru filtrarea colaborativa	12
3.10	Arhitectură de rețea neuronală pentru recomandări eficiente	12
3.11	Sistem de recomandare secvențial	13
3.12	Rețea Long-Short Term Memory	13
3.13	Adresarea problemelor Cold Start și Data Sparsity	14
3.14	Sistemul de recomandare Pinterest	14
Capitolul 4	Analiză și fundamentare teoretică	15
4.1	Sisteme de recomandare	15
4.1.1	Măsurători ale similarității	18
4.2	Procesarea limbajului natural	19
4.2.1	Procesarea simbolică	20
4.2.2	Procesarea statistică	20

4.2.3 Modelul VADER	21
4.2.4 Modelul Sentence-BERT	21
4.3 Baze de Date Graf	21
Capitolul 5 Proiectare de detaliu și implementare	24
5.1 Definirea problemei și a soluției alese	24
5.2 Arhitectura aplicației	24
5.3 Extragerea de atribute	27
5.4 Modelarea Bazei de Date	27
5.5 Sistemul de recomandare	29
5.5.1 Filtrarea bazată pe conținut	29
5.5.2 Procesarea limbajului natural în sistemul de recomandare	30
5.5.3 Descrierea produselor în sistemul de recomandare	30
5.5.4 Generarea recomandărilor	30
5.6 Inițializarea sistemului	31
Capitolul 6 Testare și validare	33
6.1 Teste unitare	33
6.2 SonarQube	34
6.3 Testare manuală	36
6.4 Evaluarea sistemului de recomandare prin interacțiunile cu utilizatorii	38
Capitolul 7 Manual de instalare	41
Capitolul 8 Concluzii	45
8.1 Dezvoltări ulterioare	46
Bibliografie	47

Capitolul 1

Introducere

1.1 Contextul proiectului

În era digitală cantitatea de informație disponibilă poate crea probleme dificile. Odată ce volumul de date de pe internet crește cu rapiditate, devine din ce în ce mai importantă oferirea de conținut relevant și personalizat utilizatorilor. Una dintre cele mai eficiente soluții pentru această problemă sunt algoritmi de obținere a informației, mai exact sistemele de recomandare.

Un sistem de recomandare este un framework computațional proiectat pentru analiza preferințelor utilizatorilor, caracteristicilor produselor și a interacțiunilor utilizator-produs pentru a genera predicții sau sugestii relevante.

Din ce în ce mai multe aplicații din diferite domenii [1] adoptă sistemele de recomandare pe zi ce trece, printre care se numără: web [2] [3], cărți [4], turism [5], muzica, televiziune și nu numai. În figura 1.1 putem observa numărul de articole științifice din diferite domenii.

Din studiile desfășurate se poate constata un trend de creștere în interesul față de sistemele de recomandare în foarte multe domenii.

Astfel, sistemul de față își propune să ofere unei aplicații web de e-commerce specializată pe vânzarea pantofilor un sistem de recomandare ca serviciu.

1.2 Importanța sistemelor de recomandare

YouTube este una dintre cele mai de succes platforme de postare a conținutului creat de utilizatori cu o cantitate enormă de conținut la dispoziția acestora, ceea ce face explorarea de videoclipuri să fie o experiență anevoioasă. Pentru a ajuta utilizatorii să găsească videoclipuri relevante dintr-un număr mare de videoclipuri, YouTube oferă un sistem de recomandare pentru a optimiza experiența utilizatorilor.

Renjie Zhou et al. [7] au întocmit un studiu 1.2 în care au obținut două seturi de date despre videoclipurile de pe YouTube folosind web crawling. În urma analizei s-a

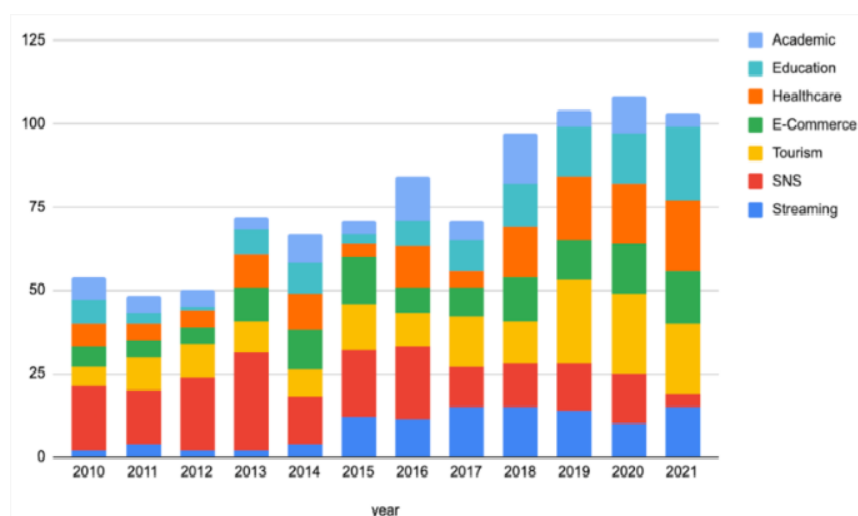


Figura 1.1: Cercetarea sistemelor de recomandare în șapte domenii din anul 2010 până în anul 2021 [6]

constatat că sistemul de recomandare a platformei este responsabil pentru 40% respectiv 60% din vizualizările videoclipurilor, concurând doar cu funcția de căutare a platformei.

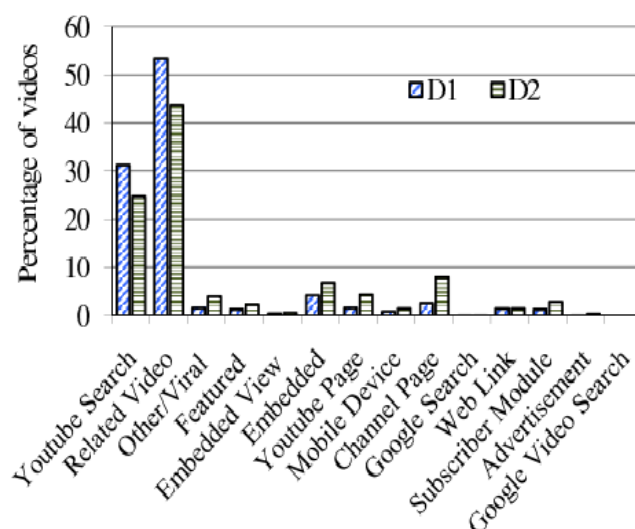


Figura 1.2:
Procentajul de videoclipuri dominat de fiecare categorie [7]

LeShop^[1] este un website de cumpărături online din Suedia care urmărește să simplifice sarcina de cumpărare a produselor de zi cu zi. Dominique Locher et al. [8], bazându-se

¹<https://www.migros.ch/en>

pe datele din figura 1.3, afirmă că acceptarea sistemelor de recomandare de către utilizatori este dată de trecerea timpului dar și de modificarea sistemului în funcție de preferințele acestora.

Ca și completare, consider că transparența sistemului de recomandare este foarte importantă. Cu cât un utilizator înțelege mai bine de ce unele produse au fost recomandate, cu atât mai mult este mai probabil să interacționeze cu acestea. Această afirmație este susținută de către Sinha et al. [9], dovedind acest fapt în urma unui studiu.

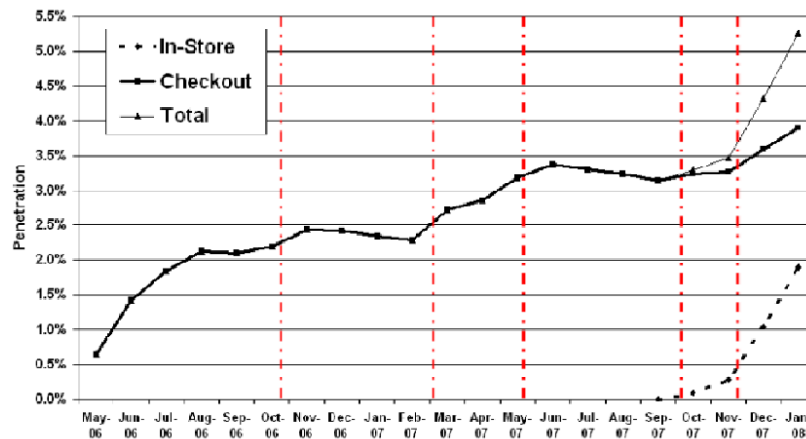


Figura 1.3:
Acceptarea sistemelor de către utilizatori [8]

O altă analiză a fost făcută pentru a obține venitul direct și indirect creat de sistemele de recomandare implementate în aplicația web LeShop, având o creștere totală de 0,50% care poate fi observată în figura 1.4.

Daniel Fleder și Kartik Hosanagar [10] examinează efectele sistemelor de recomandare asupra diversității produselor vândute. Acest lucru poate fi măsurat folosind indexul gini pentru a reflecta inegalitatea distribuției. Putem observa creșterea acestui coeficient în figura 1.5. Sistemele de recomandare folosite în analiza au fost filtrarea colaborativă bazată pe produse și bazată pe utilizatori.

Întrucât sistemele de recomandare intervin pozitiv în experiența utilizatorilor, în veniturile generate și în diversitatea conținutului, acestea devin un pilon esențial în obținerea unui avantaj față de competitori.

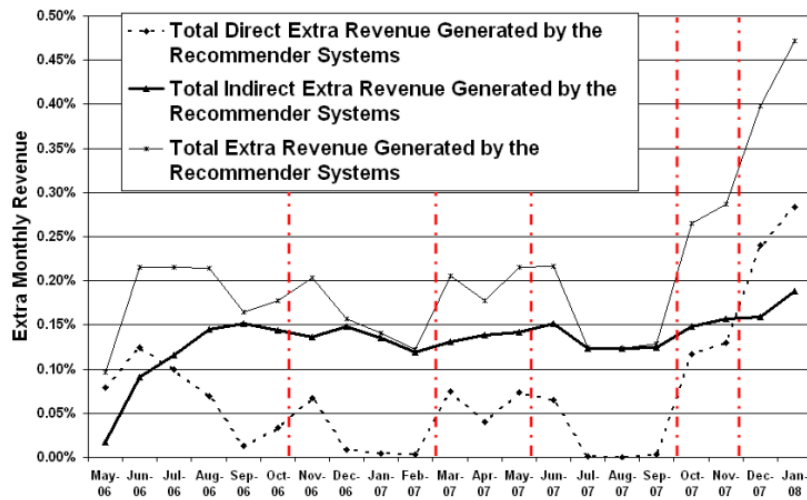


Figura 1.4:
Venitul direct și indirect generat de sistemele de recomandare [8]

	r_1	r_2
Gini before: G_0	0.533 (.045)	0.533 (.046)
Gini after: G_i	0.782 (.031)	0.567 (.047)
Difference	+0.249	+0.034
p -value	$< 10^{-6}$	$< 10^{-6}$

Figura 1.5:
Diversitatea vânzărilor oferită de sistemele de recomandare [8]

Capitolul 2

Scopul și obiectivele proiectului

2.1 Scopul proiectului

Scopul principal al acestui proiect este îmbunătățirea experienței vizitatorilor și clienților platformei. Îmbunătățirea este dată de integrarea unui sistem de recomandare care folosește algoritmi sofisticăți și date despre utilizatori pentru a oferi recomandări personalizate.

Există numeroase aspecte cheie pentru îmbunătățirea experienței utilizatorilor, printre care se numără:

- Personalizare

Prin analizarea preferințelor, istoricului tranzacțiilor și a comportamentului, sistemul oferă recomandări aliniate cu dorințele utilizatorilor.

- Descoperirea produselor

Datorită recomandărilor oferite de sistem utilizatorii vor fi expuși la un număr mai mare de produse care prin lipsa sistemului ar fi rămas nevăzute, astfel crescând calitatea interacțiunii dintre utilizator și platforma.

- Reducerea efortului de căutare

Prin reducerea efortului de căutare, ne propunem să facem experiența utilizatorilor de căutare a produselor mai plăcută. Odată ce fricțiunea introdusă de căutare este redusă, nivelul de frustrare al acestora va scădea în timp ce gradul de satisfacție va crește.

- Îmbunătățirea continuă

Sistemul urmărește îmbunătățirea recomandărilor generate prin luarea în considerare a recenziilor utilizatorilor oferite produselor cu care aceștia au interacționat.

Datorită recomandărilor personalizate oferite de sistem șansa ca utilizatorii să exprime un comportament preferențial în folosirea platformei față de platformele competitorilor va crește.

2.2 Obiectivele proiectului

Unul dintre obiectivele importante ale sistemului este creșterea ratei de conversie. Creșterea acestei metrici de performanță poate indica o experiență mai calitativă a utilizatorilor pe platformă. În direcția atingerii acestui obiectiv, o analiză de segmentare ar putea fi benefică pentru a identifica grupuri diferite de utilizatori. Sistemul de recomandare ar putea lua în considerare segmentul din care un utilizator face parte și astfel s-ar putea optimiza rata de conversie pe un astfel de segment.

Elif Ece și Keziban Orman [11] explorează performanța adusă în sistemele de recomandare de către strategia de segmentare Recency Frequency Score (RFS), unde recency este definit ca numărul de zile dintre sesiunile de cumpărături ale utilizatorului, frequency numărul de produse achiziționate și score este suma ratingurilor oferite de către utilizator produselor. Folosind datele segmentate, autorii au demonstrat că există o creștere în performanță când vine vorba de filtrarea bazată pe conținut, acest algoritm fiind la baza sistemului propus.

Scăderea timpului petrecut pe platformă a utilizatorilor care achiziționează un produs indică creștere în eficiența de căutare a unui produs relevant pentru utilizator, astfel, poate exista o creștere în satisfacția acestuia.

Deoarece nu se urmărește ca utilizatorul să fie nevoit să aleagă dintre un număr foarte mare de recomandări, sistemul va returna un număr relativ redus de produse punându-se accent pe obținerea a câtor mai multe interacțiuni utilizator-produs.

Introducerea unui număr dinamic de produse recomandate poate fi o îmbunătățire în această direcție. Numărul de produse recomandate ar putea fi influențat de nivelul implicării utilizatorului cu sistemul de recomandare. Pentru un utilizator care folosește frecvent sugestiile ar putea fi benefic un număr mai ridicat de produse recomandate, pe de cealaltă parte, pentru un utilizator care nu pare să fie înclinat spre folosirea tehnologiei un număr mai mic și cu relevanță ridicată de produse sugerate poate fi cea mai bună soluție.

Asocierea obiectivelor cu indicatorii de performanță se poate observa în tabelul 2.1

2.3 Descrierea cerințelor

Pentru a stabili cerințele sistemului, am purtat un dialog cu deținătorul platformei cu care sistemul va fi integrat. În urma interacțiunii, s-a concluzionat că în urma integrării cu sistemul platforma trebuie să afișeze o listă de recomandări personalizate pentru fiecare dintre utilizatori și de asemenea s-a pus accent pe calitatea și relevanța recomandărilor.

Cu scopul de a obține mai multe informații pentru a întocmi cerințele tehnice ale

Tabelul 2.1: Obiectivele și indicatorii de performanță a proiectului

Obiectiv	Indicator de performanță
Creșterea ratei de conversie	1% - 2% creștere în rata de conversie
Scăderea timpului petrecut pe platformă a utilizatorilor care achiziționează un produs	Timpul petrecut pe platforma de către clienți scăzut cu cel puțin 10%
Generarea de recomandări într-un timp relativ mic	Timpul necesar oferirii de recomandări să fie sub 3 secunde
Un număr substanțial dar nu copleșitor de recomandări relevante pentru fiecare utilizator	Returnarea a cel puțin 3 produse relevante pentru utilizator

sistemului a fost necesară comunicarea cu persoana care este responsabilă pentru implementarea platformei. Cerințele tehnice care reies din interacțiune sunt:

1. Există posibilitatea ca informațiile despre produse să provină din tabele html, deci sistemul va fi nevoit să extragă caracteristicile despre produse din acesta.
2. Implementarea unei interfețe REST pentru a comunica cu platforma simplu și eficient.
3. Timpul de generare a recomandărilor va trebui să fie cât mai redus pentru o experiență calitativă a utilizatorilor.
4. Produsele recomandate să fie relevante pentru utilizatorul țintă.

Capitolul 3

Stadiul actual al cercetării

În acest capitol se urmărește analiza tehnicilor folosite în dezvoltarea sistemelor de recomandare. Prin explorarea sistemelor bazate pe învățare automată, filtrare bazată pe conținut, filtrare colaborativă și metode hibride, putem obține o înțelegere amplă asupra strategiilor care duc la evoluția sistemelor de recomandare.

3.1 Algoritm de recomandare bazat pe ontologii

Peretz Shoval et al. [12] prezintă un algoritm care se mulează perfect pe recomandarea articolelor de presă electronice. Acest algoritm introduce o metodă bazată pe ontologii și pe structura lor ierarhică pentru a reprezenta profilurile utilizatorilor și produselor. Similaritatea dintre profiluri este calculată luând în considerare prezența conceptelor comune sau interconectate (luate în considerare sunt concepte bunici, părinți și copii).

Este subliniată adaptabilitatea algoritmului dincolo de domeniul din acest articol, sugerând aplicabilitatea acestuia la diverse ontologii. Extinderea potențială a metodei pentru a lua în considerare relații între concepte mai îndepărtate promite îmbunătățirea recomandărilor astfel oferind o bază solidă pentru viitoarele eforturi de cercetare.

3.2 Sistem bazat pe coeficientul de atenție

Jinbo Song et al. [13] propun un algoritm Neighbor-Aware Graph Attention Network For Recommendation care folosește date despre interacțiunile utilizator-produs structurate ca graf. Acesta începe prin folosirea unui strat de vectorizare a id-urilor utilizatorilor și produselor în spațiul latent. Următorul strat este unul de „atenție a grafului” care calculează coeficientul de atenție a nodurilor vecine bazat pe corelațiile dintre acestea și nodul țintă. Această abordare a problemei permite algoritmului agregarea vectorilor vecinilor și a nodului țintă bazat pe coeficientul de atenție.

Pentru a adresa problema complexității computaționale algoritmul folosește o metodă „max-M sampling” care preia un număr maxim „M” de vecini dintr-un subgraf. Mo-

3.3. PARADIGME CONTEXTUALE ÎN SISTEMELE DE RECOMANDARE: PREFILTRARE, POSTFILTRARE ȘI MODELARE

delul a fost evaluat folosind metrica Normalized Discounted Cumulative Gain [14] și a obținut îmbunătățiri față de alți algoritmi bazați pe grafuri cu până la 10,36%.

3.3 Paradigme Contextuale în Sistemele de Recomandare: Prefiltrare, Postfiltrare și Modelare

Gediminas et al. [15] expun în articol conceptul sistemelor de recomandare care iau în considerare contextul explicit sau implicit, astfel, autorii propun trei paradigme de integrare a contextului în recomandări: prefiltrare contextuală, postfiltrare contextuală și modelare contextuală. Prefiltrarea contextuală propune folosirea datelor a căror context este relevant iar postfiltrarea contextuală preia doar recomandări care sunt relevante în acel context. Aceste două paradigme pot suferi de contexte mult prea specifice, astfel datele pe care sistemul de recomandare le folosește vor fi uneori mult prea reduse. Autorii menționează ca fiind o soluție generalizarea contextului (de exemplu „Sâmbătă” devine „zi de weekend”).

Modelarea contextuală impune implicit folosirea unui sistem de recomandare bazat pe modele de predicție întrucât funcția de calculare a recomandării devine multidimensională. Dat fiind acest fapt, algoritmi clasici de recomandare [16] care folosesc două dimensiuni (utilizator, produs) nu pot integra o a treia dimensiune (context).

3.4 Optimizarea filtrării colaborative: LRA vs DNN

Marcel Kurovski și Florian Wilhelm [17] explorează capabilitatea algoritmilor de tip Low-Rank Aproximation (LRA) și Dense Neural Network (DNN) de a produce recomandări folosind o metodă de filtrare colaborativă. Studiul a fost făcut luând în considerare doar interacțiunile implicite și explicite între utilizator și produs, fără a lua în calcul contextul și informații demografice (unde de obicei algoritmi DNN produc recomandări de calitate superioară).

Algoritmi de tip LRA (ex. Matrix Factorization) excelează în a extrage informații ascunse din spațiul latent a matricei obținute prin produsul punctual a matricei de interacțiuni și a unei matrice de dimensionalitate mai mică. Pe de altă parte, algoritmi DNN nu sunt la fel de eficienți pentru o astfel de problemă. Autorii au arătat prin inițializarea inputurilor cu vectori din spațiul latent obținuți printr-un algoritm LRA antrenat în prealabil, că performanța algoritmilor DNN poate să o depășească pe cea a algoritmilor de tip LRA.

3.5 Sistemul de recomandare Netflix

Carlos A. Gomez și Neil Hunt [18] descriu sistemul de recomandare folosit pe platforma de streaming Netflix. Pe pagina de Home a platformei există mai multe rânduri în care conținutul este sugerat, fiecare rând corespunde unui algoritm de recomandare diferit.

Rândurile care au ca titlu genul produsului sunt date de algoritmul Personalized Video Ranker(PVA) care preia un subset din catalogul de produse bazat pe un atribut relevant (genul). Acesta îmbină personalizarea recomandărilor și popularitatea produselor pentru a ordona subsetul din catalog.

Recomandările din rândul cu titlul „Top Picks” sunt date de algoritmul Top-N Video Ranker care funcționează similar cu PVA însă acesta nu este constrâns doar la un subset din catalog.

Pentru rândul cu titlul „Trending Now” este folosit un algoritm de recomandare secvențial care folosește personalizarea și ține cont de trenduri temporale de scurtă durată (între câteva minute și câteva zile). Acesta identifică și folosește trenduri periodice cum ar fi ziua independenței sau trenduri unice de termen scurt cum ar fi un dezastru natural.

Data fiind importanța vizualizării de conținut în diferite perioade și intervale de timp, există un algoritm Continue Watching Ranker care oferă recomandări la secțiunea de „Continue watching”. Acesta folosește date cum ar fi timpul trecut de la vizualizarea precedentă, punctul la care utilizatorul a oprit vizualizarea și dispozitivele folosite. Un alt set de algoritmi se ocupă de secțiunea de căutare a platformei. Bazat pe o interogare greșită sau incompletă, aceștia returnează recomandări relevante la interogarea prezisă.

Platforma Netflix rezolvă problema de „Cold Start” [19] a membrilor noi înregistrați printr-un chestionar la intrarea inițială pe platforma. Răspunsurile din chestionar sunt folosite ca input-uri la algoritmi de recomandare, astfel oferind utilizatorilor noi sugestii relevante. Prin oferirea acestor detalii, Netflix arată o parte din complexitatea sistemului său de recomandare și își arată eforturile continue de a oferi utilizatorilor o experiență personalizată.

3.6 Rețea bayesiană pentru recomandări personalizate

Chihiro Ono et al. [20] propun un model de tip Bayesian Neural Network (BNN) pentru a rezolva două probleme: recomandări și promoții de film. Alegerea acestui model este argumentată de faptul că variabila pe care modelul o prezice poate fi schimbată, acest fapt fiind dat de dependențele dintre variabile și de distribuția probabilistică combinată a acestora.

Algoritmul menționat folosește atributele utilizatorilor, atributele filmelor, informații despre context și informații despre impresiile utilizatorilor. Datele despre impresiile utilizatorilor provin din sistemul de promovare ca feedback al utilizatorilor și sunt folosite ca input-uri pentru sistemul de recomandare. Autorii au folosit o metodă de clusterizare a atributelor după care au făcut o ierarhie între acestea. Evaluarea modelului a fost făcută între patru versiuni a acestui model și două metode clasice folosite pentru recomandări: filtrare colaborativă bazată pe utilizatori și bazată pe produse. Diferențierea dintre cele patru modele provenite din arhitectura inițială constă în tipurile de date pe care le folosesc.

Folosind metrica mean absolute error, algoritmul care folosește impresiile, atributele utilizatorilor, atributele filmelor și contextul este cu aproximativ 7,6% mai performant

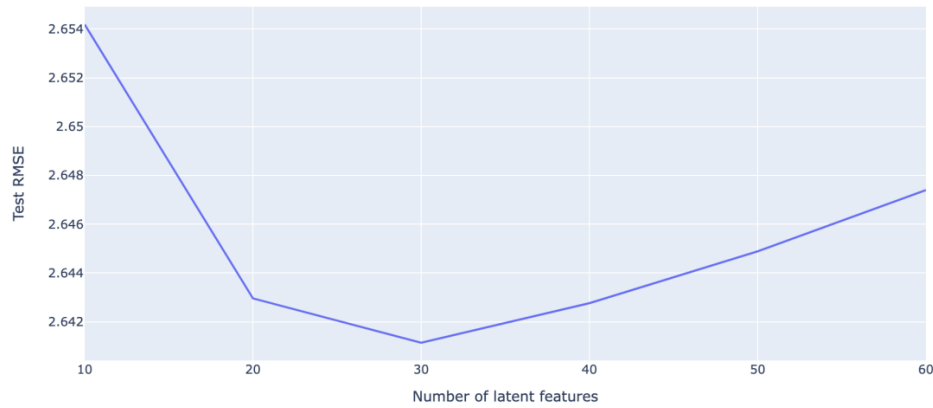


Figura 3.1: Numărul factorilor latenți [21]

decât filtrarea colaborativă bazată pe produse. Autorii explică această diferență relativ mică de performanță în proporție cu diferența de complexitate prin faptul că setul de date folosit nu este cel mai calitativ. Algoritmul propus în acest articol oferă utilizatorilor recomandări personalizate dar oferă de asemenea furnizorilor de conținut un mecanism de promovare.

3.7 Sistem de recomandare hybrid

Samin Batra et al. [21] prezintă o soluție pentru a oferi utilizatorilor platformei steam recomandări bazate pe attribute explicite (rating) și attribute implicite (timpul petrecut în joc, sentimentele din spatele review-urilor). „Alternating Least Squares” (ALS) este un model specific de „Matrix Factorization” (MF) care folosește o metodă de regresie iterativă pentru a optimiza matricile factorilor latenți corespunzători utilizatorilor și produselor. Această opțiune de implementare a sistemului de recomandare a fost aleasă pentru capabilitățile sale de a fi eficientă chiar și cu seturi de date prost calitative.

A fost explorat numărul de factori latenți folosit în procesul de antrenare a modelului, încercându-se obținerea unui rezultat mai favorabil a metricii Root Mean Squared Error.

Autorii au folosit trei abordări folosind algoritmul ALS: folosirea atributului „play-time”, folosirea atributului „playtime” și sentimentele din spatele review-urilor și atributul „playtime” alături de rating-urile date de utilizatori. Pentru prima opțiune a fost stabilit un set de criterii pentru a normaliza ratingul în funcție de atributul „playtime”, iar pentru a două, ratingul rezultat este modificat ulterior în funcție de sentimentul din spatele review-urilor. Folosind metrica „Root Mean Squared Error” aceste trei opțiuni au fost evaluate, cu 20% mai performantă față de celelalte două este cea care folosește sentimentele. Implementarea descrisă furnizează o soluție inovatoare și arată importanța tehnicilor de NLP în sistemele de recomandare.

3.8 Folosirea algoritmilor de clustering pentru popularea interacțiunilor

Problemele de “data sparsity” și scalabilitate sunt arhicunoscute în metodele de filtrare colaborativă pentru sistemele de recomandare. Song Jie Gong [22] propune în acest articol o soluție pentru a transforma matricea slab populată de interacțiuni utilizator-produs într-o matrice densă de interacțiuni. Tehnica propusă pentru rezolvarea acestei probleme este una de clusterizare folosind algoritmul nesupervizat “k means clusters”. Acesta este folosit ca mecanism de predicție pentru utilizatorii care nu au interacțiuni cu anumite produse folosind media clusterului din care face parte. Folosind același algoritm nesupervizat autorii rezolvă problema scalabilității prin clusterizarea produselor, astfel, nu sunt necesare operații la scara întregului set de date pentru a găsi vecinii unui anumit produs. După compararea unei metode clasice de filtrare colaborativă folosind metrica MAE Mean Absolute Error (MAE) algoritmul propus este mai performant cu paroximativ 8%.

3.9 Metode liniare pentru filtrarea colaborativa

Algoritmii de tip Sparse Linear Method (SLIM) învață prin aproximarea unei matrici de coeficienți agregați rezultată din matricea de interacțiuni utilizator-produs. Această metodă oferă un avantaj substanțial față de celelalte metode bazate pe MF (matrix factorization) întrucât o matrice slab populată va rezulta în operații mai puțin stresante computațional.

Yong Zheng et al. [23] subliniază două variații principale ale modelului CSLIM, CSLIM-I și CSLIM-U care folosesc filtrarea colaborativă bazată pe produse și utilizatori. Aceste variații integrează contextul în matricea coeficienților luând sau nu în considerare deviația unui rating în funcție de perechea <produs context> sau <utilizator context>. Această soluție se remarcă prin scăderea complexității computaționale și prin performanța ridicată în cazul în care există interacțiuni pentru perechea <utilizator produs> în contexte multiple.

3.10 Arhitectură de rețea neuronală pentru recomandări eficiente

Tehnicile de deep learning sunt extins folosite în sistemele de recomandare pentru abilitatea acestora de a învăța relații non-lineare între utilizatori și produse, astfel, prin algoritmul prezentat în acest articol Abebe Tegene [24] încearcă să împingă mai departe această capabilitate printr-o arhitectură aparte a rețelei neuronale.

Arhitectura are la baza două rețele neuronale în speranța de a extrage mai multe informații despre relațiile dintre utilizatori și produse. Modelul primește ca input matricea

de rating-uri din care extrage doi vectori proiectați în spațiul latent folosind un strat de încorporare, reprezentând vectorul utilizatorilor respectiv al produselor. După trecerea vectorilor prin straturile dense ale rețelei, se obține ratingul prin produsul punctual a celor doi vectori rezultați. Acest algoritm se remarcă prin rezolvarea problemei de lipsă a datelor prin tehnici de încorporare folosind rating-urile deja știute dar și prin eficiența computațională de care algoritmi clasici duc lipsă.

3.11 Sistem de recomandare secvențial

Preferințele utilizatorilor nu rămân constante de obicei, acest fapt dând naștere la conceptul de sisteme de recomandare secvențiale care încearcă să țină cont de modificarea preferințelor. Sistemul de recomandare descris de Yeongwook Yang et al. [25] folosește trei algoritmi diferiți: filtrare bazată pe conținut, Low Rank Aproximation pentru aproximarea preferinței utilizatorilor și lanțuri Markov de grad înalt.

Partea de filtrare bazată pe conținut se bazează pe termenul de frecvență-frecvență inversă a documentului (TF-IDF) pentru a folosi descrierea produselor în speranța de a adăuga informații relevante în sistemul de recomandare. Lanțurile Markov de grad înalt sunt folosite pentru a calcula probabilitatea de tranziție a unui utilizator ținând cont de interacțiunile precedente. Combinarea acestor tehnici a fost făcută prin reprezentarea matricei de probabilități într-o matrice de grad mai mic astfel mulându-se perfect cu vectorii rezultați din TF-IDF și aproximarea preferinței făcută tot cu un algoritm LRA. Sistemul reușește să îmbine armonios cele trei metode demonstrând o performanță promițătoare însă similaritatea cosinus folosită în filtrarea bazată pe conținut s-ar putea să nu fie cea mai bună soluție în calcularea similarității dintre produse în acest caz.

3.12 Retea Long-Short Term Memory

Wafa Shafqat și Yung-Cheol Byun [26] sublinează importanța datelor contextuale în sistemele de recomandare, astfel, propun un model ierarhic deep long-short term memory îmbunătățit cu date contextuale. În primul nivel al modelului acesta folosește o rețea Long-Short Term Memory (LSTM) pentru a învăța din istoricul de călătorii al utilizatorilor și a calcula probabilitățile următoarelor locații. De asemenea, la acest nivel al modelului există o unitate care extrage date contextuale despre locații. Pentru a reduce numărul de date contextuale, a fost folosită o metodă XGBoost [27] pentru a folosi doar cele mai relevante attribute.

În al doilea nivel al modelului o altă rețea LSTM a fost folosită pentru a învăța attributele contextuale încorporate cu probabilitățile locațiilor astfel rezultând un clasament al locațiilor. Această soluție remarcabilă a obținut o acuratețe de 97.2% față de 96.4% a modelului gated recurrent unit [28] și față de 94.2% a modelului LSTM bidirecțional [29].

3.13 Adresarea problemelor Cold Start și Data Sparsity

JiaoJiao Chen et al. [30] adresează problema calculării similarității dintre utilizatori în contextul sistemelor de recomandare pentru produsele din domeniul agriculturii. Pentru a calcula similaritatea dintre utilizatori, algoritmul folosește informații de bază despre aceștia (vârstă, locație, sex etc.) dar și similaritatea dintre preferințele acestora.

Problema „Cold Start” este parțial rezolvată de acest algoritm printr-un weight care oferă mai multă importanță similarității informațiilor de bază deoarece nu există date despre preferințele unui utilizator nou. De asemenea problema de lipsă a datelor este într-o oarecare măsură rezolvată prin introducerea unor parametri de „overlap” și „penalty”, astfel similaritatea preferințelor este ajustată în funcție de lipsa preferințelor comune dintre cei doi utilizatori.

Acest articol propune o soluție fiabilă la problemele întâmpinate de acest tip de algoritmi, însă complexitatea computațională poate împiedica folosirea acestuia pentru recomandări în timp real. Problema de „Cold Start” pentru produsele proaspăt introduse în sistem încă există, deoarece nici un utilizator nu a interacționat cu acestea.

3.14 Sistemul de recomandare Pinterest

Rex Ying et al. [31] prezintă un model Graph Convolutional Network (GCN) care este adecvat pentru sistemele de recomandare care folosesc date structurate ca graf. Dat fiind faptul ca modelul este folosit pe siteul Pinterest care are o cantitate impresionantă de date în folosință, scalabilitatea acestui model este foarte importantă. În contrast cu alte rețele GCN [32] care fac convoluții multiplicând matricea de atribute cu puteri ale matricii Laplacian, modelul propus aplică convoluții pe vecinătatea unui nod. O altă tehnică pentru a oferi o mai bună scalabilitate este un random-walk [33] scurt pentru a selecta subgraful pe care convoluțiile vor fi aplicate.

O acțiune importantă a convoluțiilor pe grafuri este agregarea informațiilor despre atribute într-o vecinătate a grafului. A fost introdusă o greutate bazată pe similaritatea random walk în funcția de agregare care a dus la o performanță cu 46% mai mare în testele offline. Folosind metrica Mean Reciprocal Rank (MRR) care poate fi interpretată ca fiind media rangului a celei mai relevante recomandări, funcția de agregare prezentată a adus o valoare de 0.59 față de 0.51 a funcției mean pooling și 0.31 a funcției max pooling.

Capitolul 4

Analiză și fundamentare teoretică

În acest capitol se vor stabili bazele teoretice ale elementelor folosite în procesul de proiectare și implementare a sistemului propus.

4.1 Sisteme de recomandare

S. Shaik Salam et al. [34] definesc un sistem de recomandare ca fiind un model care are ca scop prezicerea preferințelor unui utilizator și care face sugestii bazându-se pe acestea.

Cazul de utilizare [35] obișnuit a unui sistem de recomandare este într-un mediu de retail. Un utilizator vizitează o librărie online unde pagina de pornire listează cele mai vandute produse și, de asemenea, o listă cu articole recomandate. Această listă ar putea include, de exemplu, o carte nouă publicată de unul dintre autorii preferați ai utilizatorului și o carte de bucate a unui nou autor. Dacă utilizatorul va găsi aceste sugestii utile sau nu, funcția de recomandare personalizată va fi responsabilă pentru acest fapt.

Toate sistemele de recomandare au același scop, de a oferi utilizatorilor o experiență personalizată, însă îl ating în moduri diferite. Una din primele metode ale acestor sisteme a fost filtrarea colaborativă [16] care folosește semnale colaborative dintre utilizatori și produse.

Această metodă oferă două metode inițiale, una bazându-se pe similaritatea dintre utilizatori (figura 4.1) (filtrarea colaborativă bazată pe produse) dată de formula :

$$R_{ui} = \frac{\sum_{j \in N(u)} (R_{uj} \cdot \text{similarity}(u, j))}{\sum_{j \in N(u)} |\text{similarity}(u, j)|}$$

Unde:

- R_{ui} este ratingul prezis a produsului i de utilizatorul u

- R_{uj} este ratingul prezis a produsului i de utilizatorul j
- $N(u)$ este setul de utilizatori similari cu utilizatorul u
- $similarity(u, j)$ este o măsură a similarității dintre utilizatorul u și utilizatorul j .

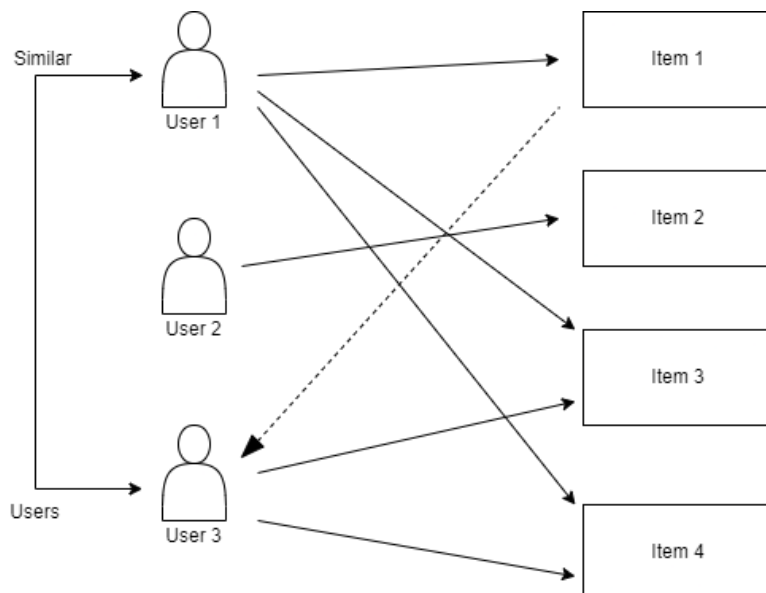


Figura 4.1:
Filtrarea colaborativă bazată pe utilizatori

A doua este bazată pe similaritatea dintre produse (figura 4.2) (filtrarea colaborativă bazată pe utilizatori) care este reprezentată de formula :

$$R_{ui} = \frac{\sum_{j \in I(u)} (R_{uj} \cdot similarity(i, j))}{\sum_{j \in I(u)} |similarity(i, j)|}$$

Unde:

- R_{ui} este ratingul prezis a produsului i de utilizatorul u
- R_{uj} este ratingul prezis a produsului i de utilizatorul j
- $I(u)$ este setul de produse cu care utilizatorul u a interactionat
- $similarity(i, j)$ este o masura a similarității dintre produsul i și produsul j .

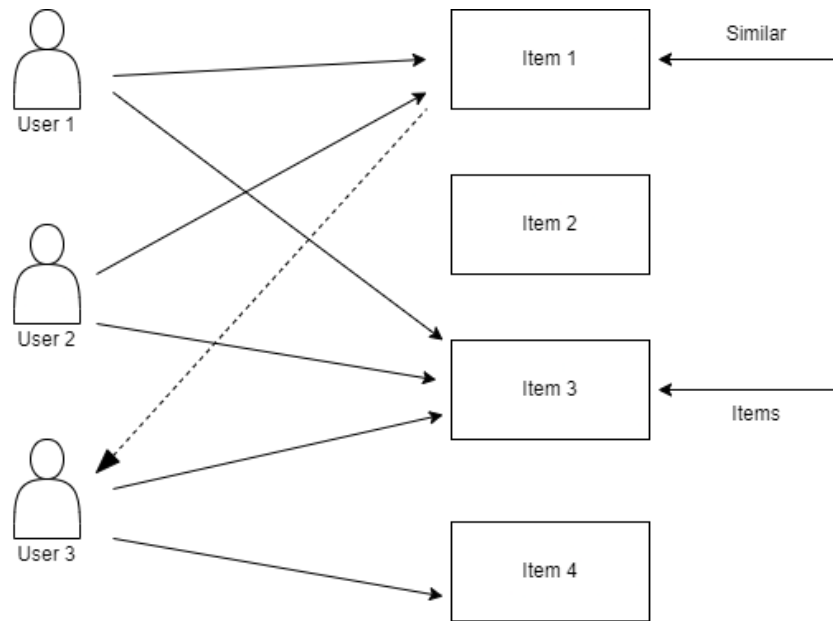


Figura 4.2:
Filtrarea colaborativă bazată pe produse

Un alt algoritm utilizat este filtrarea bazată pe conținut [36] care produce recomandări bazându-se pe caracteristicile produselor și pe preferințele obținute din interacțiunile anterioare ale utilizatorilor cu produsele. Prin această metodă (figura 4.3) produsele sunt reprezentate printr-un profil al caracteristicilor acestora. Un profil al utilizatorilor este format pornind de la preferințele acestora spre diferite produse iar recomandările rezultă din compararea similarității dintre profilele produselor cu care acesta nu a interacționat și profilul utilizatorului respectiv.

Sistemele hibride de recomandare [37] combină atuurile tehnicilor de recomandare pentru a oferi utilizatorilor recomandări mai precise și personalizate. Aceste sisteme urmăresc să depășească limitările metodelor individuale de recomandare, cum ar fi filtrarea colaborativă și filtrarea bazată pe conținut. De exemplu, filtrarea colaborativă are o problemă "Cold Start" când vine vorba de produse noi intrate în sistem respectiv filtrarea bazată pe conținut nu produce recomandări relevante pentru un utilizator nou în sistem întrucât nu deține date despre interacțiunile sale cu alte produse. Prin combinarea diferitelor abordări, sistemele hibride pot oferi o soluție mai robustă și mai eficientă.

Unele sisteme hibride urmăresc să folosească o nouă dimensiune pentru a produce recomandări mai relevante pe lângă dimensiunea utilizator și produs. Acea dimensiune este contextul în care un utilizator a interacționat cu acel produs astfel dovedind o performanță mai mare a recomandărilor față de alte metode clasice [38].

Sistemele de recomandare bazate pe modele folosesc tehnici de învățarea automată pentru a face recomandări personalizate. Aceste sisteme creează modele de predicție sau clasare bazate pe datele utilizatorului și pe diverse date cum ar fi cele contextuale,

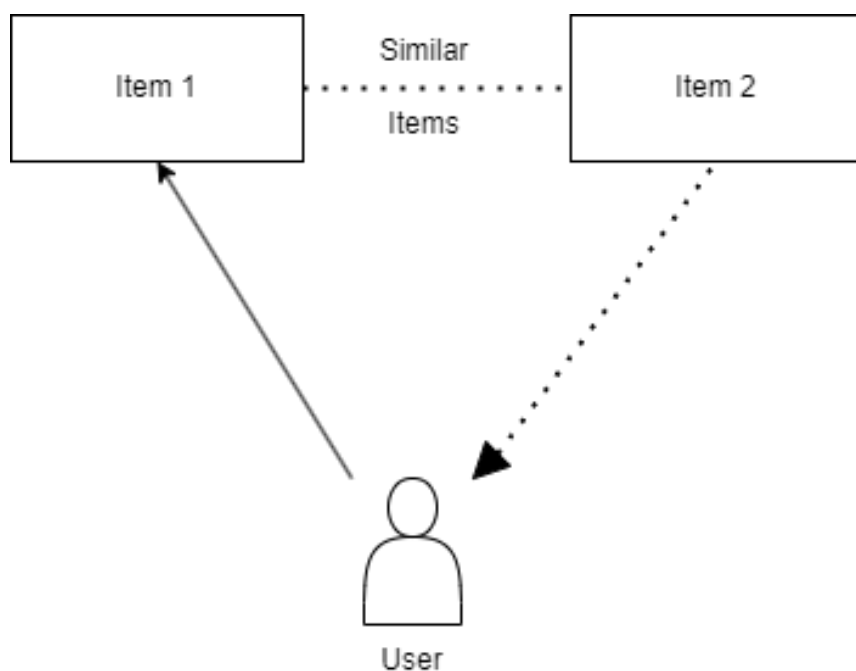


Figura 4.3:
Filtrarea bazată pe conținut

secvențiale sau ale produselor. Aceste sisteme au capacitatea de a gestiona foarte bine lipsa datelor, o problema de care metodele clasice suferă iar performanța lor primește un impact negativ datorită acestui fapt. De asemenea, aceste metode se pot adapta foarte bine la schimbarea în preferințele utilizatorilor.

4.1.1 Măsurători ale similarității

Similaritatea cosinus ¹ reprezintă valoarea cosinus a unghiului dintre doi vectori fără a lua în calcul magnitudinea acestora. Această măsurătoare este dată de formula :

$$S_C(A, B) = \frac{\sum_{i=1} A_i \cdot B_i}{\sqrt{\sum_{i=1} A_i^2 \cdot \sum_{i=1} B_i^2}} \quad (4.1)$$

Unde:

- $S_C(A, B)$ este similaritatea cosinus dintre vectorul A și vectorul B
- A_i respectiv B_i sunt componente ale acestor vectori.

¹Similaritatea cosinus https://en.wikipedia.org/w/index.php?title=Cosine_similarity_&oldid=1169535260

În contextul sistemelor de recomandare acești vectori pot fi reprezentați de vectorii de interacțiuni dintre 2 utilizatori și același set de produse. În filtrarea bazată pe conținut vectorii pot fi reprezentați de listele care reprezintă caracteristicile a două produse.

Corelația Pearson² măsoară corelația lineară dintre două variabile, în cazul filtrării colaborative reprezintă relația lineară dintre ratingurile date de către utilizatori și este dată de formulă:

$$\text{pearson_correlation}(u, v) = \frac{\sum_i (R_{ui} - \overline{R_u}) \cdot (R_{vi} - \overline{R_v})}{\sqrt{\sum_i (R_{ui} - \overline{R_u})^2} \cdot \sqrt{\sum_i (R_{vi} - \overline{R_v})^2}} \quad (4.2)$$

Unde:

- R_{ui} și R_{vi} sunt ratingurile date de utilizatorul u respectiv v la același produs
- $\overline{R_u}$ și $\overline{R_v}$ sunt mediile ratingurilor userului u și v

Similaritatea Jaccard³ măsoară similaritatea dintre două mulțimi folosind intersecția și uniunea acestora. În sistemele de recomandare aceasta poate fi folosită pentru a calcula similaritatea dintre două mulțimi de produse cu care utilizatorul a interacționat și este dată de formulă :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (4.3)$$

Unde:

- $J(A, B)$ este similaritatea Jaccard dintre mulțimea A și B

4.2 Procesarea limbajului natural

Procesarea limbajului natural (NLP)⁴ are ca obiectiv principal de a da abilitatea calculatoarelor să înțeleagă conținutul documentelor, ceea ce include și nuanțele semantice ale acestora. O dată atins acest obiectiv sistemul poate extrage informații, de asemenea poate clasifica și organiza documente. Această ramură a inteligenței artificiale a trecut prin mai multe etape. În fiecare dintre acestea abordarea problemei s-a schimbat, având ca rezultat îmbunătățirea sistemelor.

²Corelația Pearson https://en.wikipedia.org/w/index.php?title=Jaccard_index&oldid=1175671188

³Similaritatea Jaccard https://en.wikipedia.org/w/index.php?title=Jaccard_index&oldid=1175671188

⁴Procesarea limbajului natural https://en.wikipedia.org/w/index.php?title=Natural_language_processing&oldid=1175983943

4.2.1 Procesarea simbolistică

Această abordare [39] se rezumă la a oferi sistemului un set de reguli bine definite sau un sistem de cunoștințe. Procesarea simbolistică se aseamănă foarte mult cu modul în care oamenii învață o limbă, prin reguli gramaticale bine definite și o sintaxă cu o valoare semantică explicită.

Un avantaj pe care sistemele bazate pe modele de învățare automată nu o au în prezent este transparența și abilitatea de a explica rezultatele. Prin folosirea unor cunoștințe lingvistice și reguli este mult mai ușor de a înțelege cum sistemul ajunge la o anumită interpretare sau concluzie. Această transparență este folositoare în analiza documentelor legale și tehnice [40].

Metoda simbolistică are și dezavantaje totuși, în momentul în care corpul de text analizat conține un anumit nivel de complexitate și ambiguitate, ceea ce se întâmplă des, sistemul nu mai este la fel de capabil să performeze la fel. Înțelegerea semanticii dată de expresii, metafore sau de diferite contexte devine o sarcină foarte costisitoare, acesta fiind un motiv pentru care metodele bazate pe simbolistică sunt folosite împreună cu alte metode pentru a îmbunătăți rezultatele sistemului.

4.2.2 Procesarea statistică

Cu trecerea anilor, regulile bine definite din procesarea simbolistică au fost înlocuite de reguli statistice⁵ bazate pe modele de învățare automată, în consecință, mai puține reguli generalizate au înlocuit mai multe reguli detaliate.

Această metodă de procesare abordează dintr-un punct de vedere probabilistic problema procesării, a înțelegerii și a generării de limbaj natural. Problemele cum ar fi cuvinte sau expresii cu mai multe interpretări, metafore sau înțelegerea unui cuvânt bazată pe un context anume devin probleme soluționabile.

Procesarea statistică adoptă o varietate de tehnici, cea mai influentă în anii recenti fiind rețele neuronale cu arhitectură Transformer [41]. Această arhitectură înlocuiește straturi recurente și straturi convoluționale cu un sistem de atenție. Mecanismul de atenție oferă posibilitatea modelului de a modifica importanța fiecărui element din secvența de inputuri astfel fiind capabil să rețină dependențe și relații.

Deși este o paradigmă revoluționară în procesarea limbajului natural, procesarea statistică are și dezavantaje :

- modelele necesită o cantitate foarte mare de date pentru a fi antrenate
- intensive computațional
- reflectarea opiniilor subiective în rezultatele modelelor

⁵Procesarea statistică <https://academic.oup.com/jamia/article/18/5/544/829676?login=false>

4.2.3 Modelul VADER

Modelul VADER (Valence Aware Dictionary and Sentiment Reasoner) [42] este un model bazat pe reguli pentru a extrage sentimentele din spatele unui corp de text din spațiul rețelelor sociale, însă nu este singurul domeniu în care acesta excelează. C.J. Hutto și Eric Gilbert au agregat mai multe dicționare cum ar fi SentiWordNet și ANEW pentru a obține un număr mai mare de date. Pe lângă această agregare dicționarul a fost îmbogățit cu argoul din spațiul rețelelor sociale, acesta incluzând expresii ca "lol" și emoticoane.

Un dezavantaj ale dicționarelor folosite de autori este că nu specifică intensitatea sentimentelor. Pentru a remedia acest neajuns autorii au folosit paradigmă "wisdom-of-the-crowd" pentru a îmbogăți dicționarul cu intensitatea sentimentelor. 10 oameni au fost responsabili pentru evaluare, ceea ce a fost dovedit ca fiind mai eficient de cât evaluarea unui singur specialist în domeniu. Modelul Vader are rezultate similare cu alte modele de învățare automată dar se evidențiază prin viteza computațională a acestuia dar și prin transparență datorită faptului că este o abordare simbolistică.

Un neajuns care necesită atenție este detecția sarcasmului în propoziții. Întrucât acest model se bazează pe un dicționar, ironia și sarcasmul pot rămâne nedetectate. Ținând cont de acest fapt, integrarea acestui model în sistemul de recomandare are nevoie de considerare atentă întrucât unele persoane pot oferi comentarii ironice produselor.

4.2.4 Modelul Sentence-BERT

Nils Reimers și Iryna Gurevych [43] prezintă în articolul publicat rețeaua neuronală SBERT care este o variație a modelului BERT. SBERT este proiectat pentru reprezentarea propozițiilor într-un spațiu vectorial pentru sarcini de calculare a similarității și nu numai.

Spre deosebire de BERT care primește o singură propoziție ca input, SBERT folosește o arhitectură de rețele neuronale siameze, astfel primind multiple propoziții ca input. Aceste două rețele din componența modelului au greutăți antrenabile comune. Pentru sarcina de calculare a similarității SBERT a fost antrenat folosind funcția de pierdere Mean Squared Error care este folosită pentru a reduce eroarea dintre similaritatea prezisă și cea inițială. Astfel, rețeaua neuronală învață să reprezinte propozițiile în niște vectori care încorporează mai bine informațiile despre propoziții.

Această optimizare a modelului deja existent oferă îmbunătățiri față de majoritatea soluțiilor state-of-the-art nu doar la încorporarea propozițiilor în vectori dar și când vine vorba de viteză și eficiența computațională a modelului.

4.3 Baze de Date Graf

O baza de date graf este [44] un sistem de management a unei baze de date care suportă operații de Create, Read, Update și Delete expuse asupra unui model a datelor sub formă unui graf. Acestea sunt optimizate pentru performanța tranzacțiilor. Există

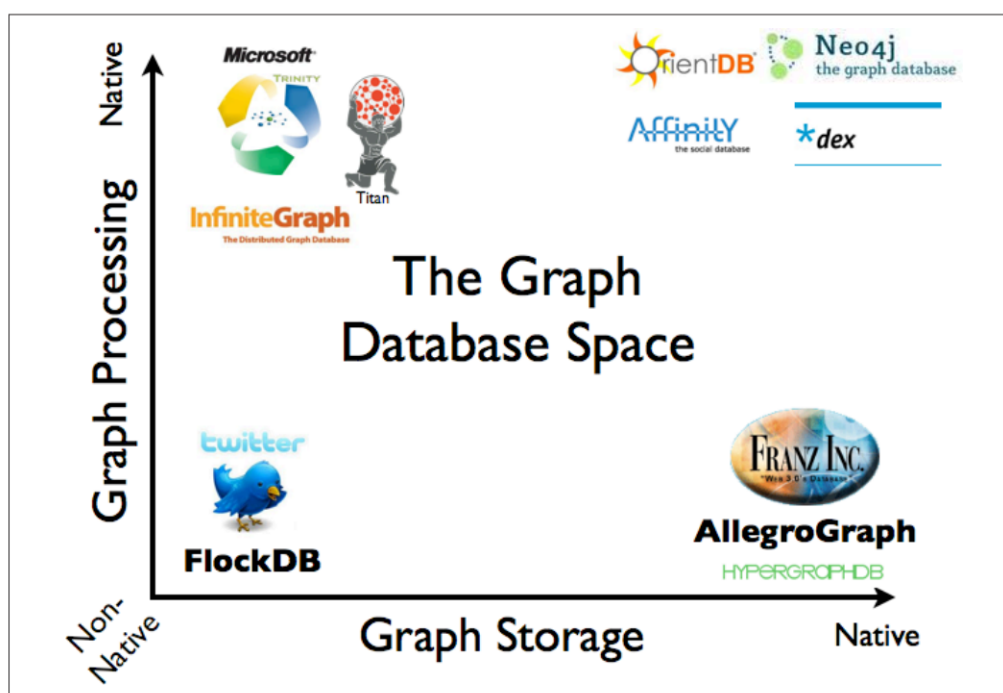


Figura 4.4: Spațiul bazelor de date graf în funcție de procesare și stocare [44]

două proprietăți de care ar trebui să ținem cont când vine vorba de a alege o astfel de tehnologie:

- **Tipul de stocare**
Unele baze de date de tip graf folosesc tipul de stocare a datelor nativ-graf. Alte tehnologii serializează datele într-o baza de date relațională sau orientată pe obiecte.
- **Motorul de procesare**
Motorul de procesare nativ a bazelor de date graf se bazează pe adiacență fără nevoia unui index. Acest concept se referă la abilitatea nodurilor de a cunoaște vecinii fără folosirea unei structuri de date complexe.

Un motiv foarte bun pentru alegerea unei astfel de tehnologii este creșterea performanței față de bazele de date relaționale și NoSQL în cazul datelor foarte conectate. Față de bazele de date relaționale unde performanța interogărilor de tip join scade în timp ce volumul de date crește, performanța bazelor de date graf tinde să rămână constantă. Acest lucru se datorează interogărilor care sunt localizate pe o singură parte a grafului. Timpul de execuție este proporțional cu mărimea grafului care satisface interogarea.

Bazele de date graf sunt foarte flexibile întrucât adăugarea a noi tipuri de noduri (entități) și a noi tipuri de relații nu afectează funcționalitățile originale. Datorită acestui fapt, nu este necesară o modelare conceptuală înaintea implementării foarte îndelungată

spre deosebire de bazele de date relaționale care sunt mult mai rigide. Această flexibilitate aduce avantaje și la costurile necesare pentru modificarea bazei de date în cazul schimbărilor în cerințele aplicației.

Neo4j este o baza de date graf care urmărește să asigure consistență și toleranță la partiționare (sistem CP). Operațiile de scriere în acest sistem se execută într-un context tranzacțional, cu operațiile de scriere blocante pentru a asigura consistența datelor. Odată ce tranzacția a fost executată cu succes, datele sunt mutate din memorie pe disc pentru a fi durabile. În cazul în care se încearcă executarea a două sau mai multe tranzacții Neo4j detectează posibilitatea unui deadlock, astfel asigurându-se că tranzacțiile sunt executate secvențial.

Neo4j suportă echilibrarea sarcinii asupra multiplelor instanțe într-un cluster. De obicei, folosește o topologie write-master cu read-slaves. De la versiunea 1.8, Neo4j suportă operații de scriere prin instanțe slave. Deși prezintă această opțiune, scrierea pe astfel de instanțe necesită sincronizarea cu instanța master înainte de trimiterea răspunsului spre client. Acest pas în plus deși oferind o durabilitate crescută poate aduce încetinirea operațiilor.

Capitolul 5

Proiectare de detaliu și implementare

Capitolul acesta cuprinde implementarea sistemului propus, construit pe baza teoretică menționată în capitolele anterioare.

5.1 Definirea problemei și a soluției alese

În vremurile noastre volumul de produse, servicii, conținut și nu numai poate deveni copleșitor foarte repede. Pornind de la această problema, am propus un sistem de recomandare bazat pe conținut care oferă utilizatorilor sugestii personalizate și țintite spre preferințele acestora. De asemenea, sistemul folosește descrierea produselor în procesul de recomandare cu scopul creșterii calității recomandărilor. Pentru a oferi altor aplicații o integrare mai ușoară cu sistemul de recomandare a fost implementat o interfață REST pentru comunicare.

5.2 Arhitectura aplicației

A fost aleasă o arhitectură de tipul client-server întrucât această oferă câteva avantaje. Sistemul de recomandare este decuplat de aplicația client astfel sistemul ar putea fi folosit de orice fel de aplicație atâta timp cât i se oferă datele necesare.

O arhitectură a unui astfel de sistem constă în mai multe module fiecare cu un rol specific. Partea de backend a fost implementată folosind python versiunea 3.11.5 care primește de la client în prima parte datele despre utilizatori și despre produse după care sistemul va fi pregătit să facă recomandări pentru un anumit utilizator la cererea aplicației.

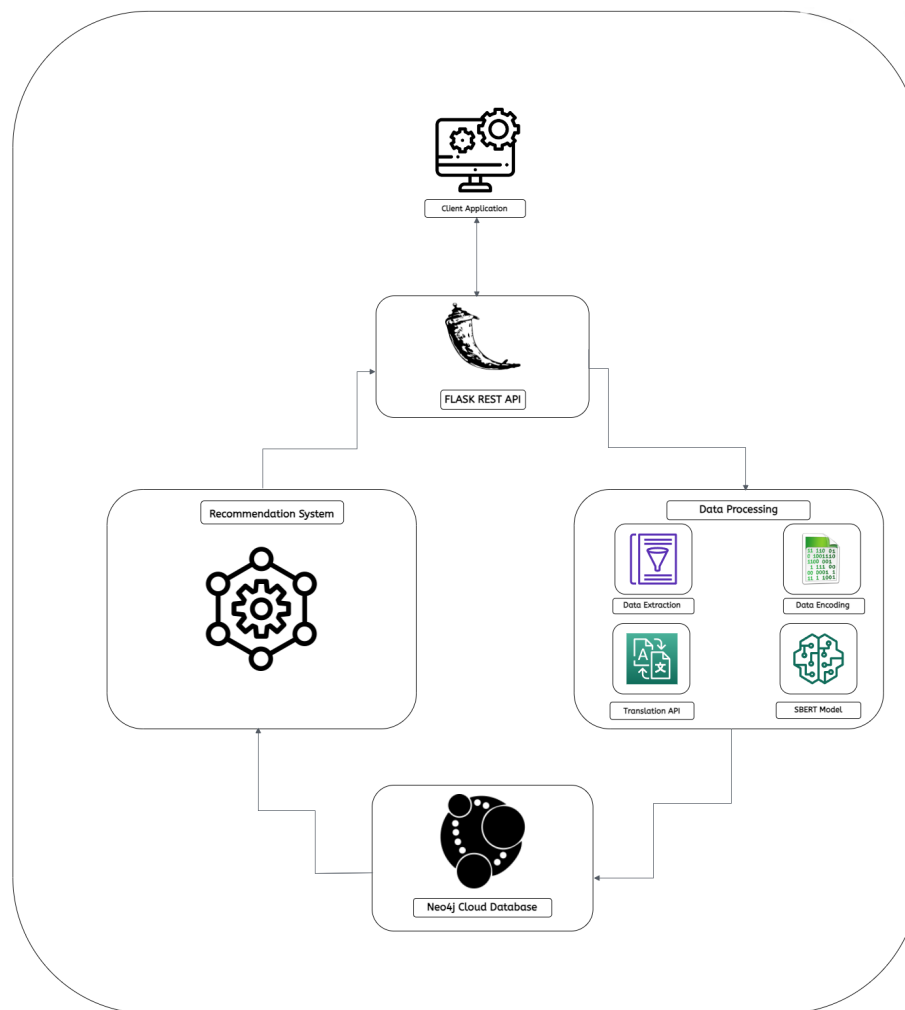


Figura 5.1: Arhitectura sistemului

Componentele software au fost organizate folosind pattern-ul MVCS întrucât se separă preocupările în componente distincte **Model**, **View**, **Controller** și **Service**. **Modelul** încapsulează datele și logica aplicației, promovând integritatea și reutilizarea acestora. **Controllele** acționează ca intermediari, gestionând intrarea în sistem și orchestrând interacțiunile dintre **Model** și **View**. Includerea componentelor **Service** decuplează și mai mult logica de **Controller**, promovând o mai bună organizare a codului.

În tabelele 5.1 și 5.2 sunt prezentate tehnologiile și librăriile folosite în dezvoltarea sistemului.

Tehnologie	Rol
Python	Limbaaj de programare care oferă vaste librării orientate spre data science și machine learning
Neo4j	Bază de date graf care oferă performanță și flexibilitate în cazul datelor puternic conectate
SBERT	Rețea neuronală de tip transformers care transpune text într-un spațiu vectorial
Vader	Model bazat pe reguli care folosește un dicționar vast pentru analiza sentimentelor

Tabelul 5.1: Tehnologii

Librărie sau Framework	Rol
Neomodel	OGM (Object Graph Mapper) pentru maparea obiectelor la noduri
Re	Oferă un motor Regex pentru manipularea textului
Dotenv	Posibilitatea de salvare și încărcare a variabilelor de mediu
Pytest	Testarea unitară a sistemului
Flask	Implementarea interfeței REST pentru sistemul de recomandare
Sklearn	Calcularea similarității cosinus
Flask_Cors	Configurarea CORS

Tabelul 5.2: Tehnologii

5.3 Extragerea de attribute

Sistemul de recomandare are abilitatea de eventuala preluare a datelor despre produse sub formă de element/tabel html. Modulul responsabil de această prelucrare folosește expresii regulate (Regex).

O funcție din modul este responsabilă pentru extragerea descrierii produsului într-un câț această poate oferi o altă înțelegere asupra interacțiunilor dintre utilizator și produsul respectiv.

Expresia regulată din relația 5.1 constă din două grupuri de captare. Primul grup `(.*?)` captează conținutul inclus în etichetele `<td>`. Datorită cuantificatorului non-lacom acesta se potrivește cu orice caracter de zero sau de mai multe ori, dar de cat mai puține ori posibil. Al doilea grup de captare captează conținutul din etichetele `<strong>`, utilizând din nou cuantificatorului non-lacom pentru a capta textul între `<strong>` și `</strong>`. Aceste grupuri de captare permit extragerea ușoară a datelor specifice din fiecare celulă a tabelului HTML, facilitând organizarea acestora într-o structură de date pentru prelucrare ulterioară.

```
<td.*?>(.*?)</td><td.*?><strong>(.*?)</strong></td>
```

Listing 5.1: Regex folosit pentru extragerea caracteristicilor

5.4 Modelarea Bazei de Date

Pentru algoritmul propus s-a folosit o bază de date graf Neo4J¹. Două dintre caracteristicile foarte importante a acestora este eficiența traversării între noduri și viteza acestora în cazuri în care există foarte multe relații, ceea ce face acest tip de baza de date perfectă pentru un sistem de recomandare.

În cazul actual, au fost identificate două tipuri de entități care au devenit noduri: produsele și utilizatorii.

Nodurile care descriu utilizatorii au următoarele attribute:

- `id (String)` : identificator generat automat de baza de date
- `client_id (Integer)` : identificatorul utilizatorului dat de aplicația client
- `name (String)` : necesar doar pentru identificarea mai ușoară a nodurilor la testare
- `profile (List<Float>)` : lista care conține preferințele utilizatorului
- `favorite_description (List<Float>)` : lista care conține valorile încorporate ale descrierii unui produs

¹Neo4j White Paper
Innovation_EN_US.pdf

https://go.neo4j.com/rs/710-RRC-335/images/Neo4j_WP_Retail_Innovation_EN_US.pdf

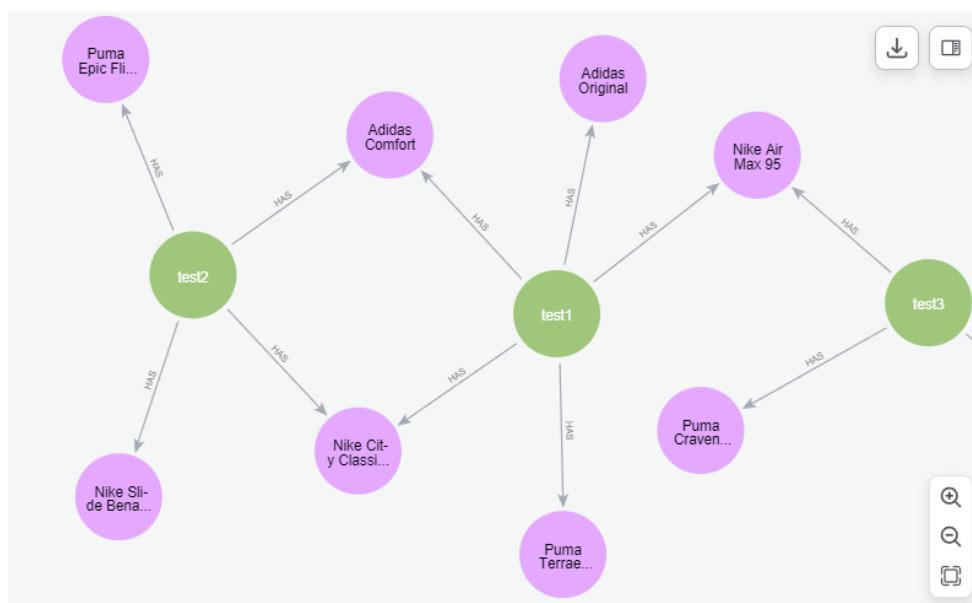


Figura 5.2: Entități din baza de date

Nodurile care descriu produsele au următoarele atribute:

- id (String) : identificator generat automat de baza de date
- client_id (Integer) : identificatorul utilizatorului dat de aplicația client
- color (String) : descrie culoarea produsului
- type (String) : descrie tipul produsului
- brand (String) : descrie producătorul produsului
- profile (List<Integer>) : listă care descrie caracteristicile produsului în spațiul vectorial
- embedding (List<Float>) : listă care conține valorile încorporate ale descrierii produsului

Există un singur tip de relație între nodurile utilizator și produs, aceea fiind "HAS". Deși există un singur tip de relație, diferite instanțe a aceluiași tip de relație poate avea proprietăți diferite. Relațiile dispun de proprietatea "action" de tip String care semnifică tipul interacțiunii având următoarele valori posibile: "REVIEWD", "SEEN", "BOUGHT" și "WISHLISTED". Un alt atribut al relațiilor este "weight" de tip Float care are valori în intervalul [0,1] și semnifică greutatea interacțiunii în procesul de calculare a recomandărilor.

Ian Robinson et al. ridică un semnal de alarmă la modelarea entitatilor ca relații. Ar putea exista o problemă la modelarea review-urilor în anumite cazuri unde acestea

sunt modelate ca muchii, dar alte relații între review și diferite entități ar fi necesare. În cazul nostru nu există acest impediment deoarece sistemul nu va avea nevoie de review ca entitate.

5.5 Sistemul de recomandare

În acest subcapitol se va prezenta modul de implementare a algoritmului de filtrare bazată pe conținut.

Algoritmul de față are un atu, acela că nu suferă de problema cold start în cazul produselor. Spre deosebire de filtrarea colaborativă care nu poate oferi recomandări când un produs nou este introdus în sistem, filtrarea bazată pe conținut poate compara produsele fără ca acestea să aibă o interacțiune cu un utilizator. Un impediment pentru acest sistem este cazul în care există utilizatori noi care nu au interacționat cu produsele. Pentru a oferi o soluție am propus recomandările bazate pe popularitatea produselor pentru utilizatorii intrați recent în sistem.

În contextul acestui sistem popularitatea se referă la numărul de muchii spre un nod utilizator a unui nod produs.

5.5.1 Filtrarea bazată pe conținut

La baza algoritmului [4.3](#) stă compararea între atributele produselor cu care utilizatorul a interacționat și atributele celorlalte produse. Pentru a împlini această sarcină este necesară folosirea metodei one hot encoding pentru a mapa caracteristicile fiecărui produs într-un vector reprezentativ care poate fi comparat, astfel trecând de la date categorice la date numerice. Rezultatul obținut în urmă aplicării acestei metode va fi un vector de lungimea tuturor valorilor unice a atributelor în care o valoare egală cu 1 indică prezența unei caracteristici.

Odată ce profilele produselor au fost calculate, următorul pas major este crearea profilelor utilizatorilor care se bazează pe interacțiunile dintre aceștia și produse. Crearea profilului utilizatorului este dată de următoarea formulă:

$$v_u = \frac{\sum(w_i \cdot v_i)}{t_i}$$

unde v_u este profilul utilizatorului, v_i este profilul produsului, w_i este greutatea interacțiunii și t_i este numărul de produse cu care utilizatorul a interacționat.

Profilul utilizatorului va conține preferințele sale pentru fiecare atribut pe care un produs îl poate avea. Profilele utilizatorilor și a produselor vor fi salvate în baza de date ca proprietăți a nodurilor respective.

5.5.2 Procesarea limbajului natural în sistemul de recomandare

Pentru a crește relevanța recomandărilor returnate de sistem greutatea relației review este modificată în funcție de sentimentul din spatele textului utilizatorului. Analiza sentimentului a fost făcută cu ajutorul algoritmului VADER [4.2.3](#) (Valence Aware Dictionary and Sentiment Reasoner) care folosește un dicționar construit în prealabil pentru a calcula un coeficient de valență pe baza căruia sentimentul este ales.

În funcție de sentiment rating-ul review-ului este schimbat, de exemplu pentru un rating de 3 și un sentiment foarte pozitiv ratingul final va fi 4 cu o greutate de 0.8. În cazul în care ratingul este 3 și sentimentul prezent este unul puternic negativ ratingul devine 2 cu o greutate de 0.3. Prin urmare, în cazul în care sentimentul recenziei este pozitiv ratingul va crește cu 1, respectiv din acesta se va scădea valoarea 1 în cazul în care sentimentul este negativ. În cazul în care sentimentul este neutru valoarea ratingului nu va fi schimbată.

Prin această tehnică chiar dacă un utilizator supraestimează sau subestimează interacțiunea sa cu un produs sistemul va fi capabil într-o oarecare măsură să corecteze importanța oferită interacțiunii.

5.5.3 Descrierea produselor în sistemul de recomandare

Întrucât descrierea produselor poate conține informații contextuale care ar putea fi relevante s-a urmărit introducerea descrierilor în sistemul de recomandare. Pentru a transpune textul descrierilor într-un spațiu vectorial pentru a le putea compara a fost folosit modelul all-mpnet-băse-v2 [4.2.4](#) descris în partea teoretică.

Am creat o clasă singleton care se ocupă de inițializarea modelului la pornirea sistemului. Aceasta are o metodă care primește un paragraf și returnează o listă de liste de valori încorporate, câte o listă pentru fiecare propoziție. Pentru a obține o singură valoare pentru fiecare propoziție am optat pentru calcularea mediei tuturor valorilor din reprezentarea vectorială a unei propoziții, astfel obținând o listă a cărei index reprezintă poziția propoziției din paragraf.

Fie P un paragraf care conține n propoziții, unde fiecare propoziție S_i are m valori încorporate notate ca $E_{i,j}$ unde $j=1, 2, \dots, m$.

Astfel obținem funcția :

$$\text{avg_embedding}(P) = \left(\frac{1}{m} \sum_{j=1}^m E_{1,j}, \frac{1}{m} \sum_{j=1}^m E_{2,j}, \dots, \frac{1}{m} \sum_{j=1}^m E_{n,j} \right)$$

5.5.4 Generarea recomandărilor

Primul pas în generarea de recomandări este ca sistemul să interogheze baza de date pentru a returna toate produsele cu care utilizatorul nu a interacționat. Următorul pas este calcularea similarității cosinus [4.1.1](#) dintre profilul utilizatorului și profilul produselor, astfel, similaritatea cea mai mare indică cel mai relevant produs pentru utilizator.

Cu gândul de a introduce descrierile în generarea de recomandări a fost luată în considerare descrierea produsului cu care utilizatorul are cea mai "puternică" relație. Această descriere va fi folosită pentru compararea cu descrierile celorlalte produse obținând similaritatea cosinus dintre ele.

Pentru a agrega similaritatea profilelor cu similaritatea descrierilor este calculată o medie ponderată între acestea, similaritatea profilelor având o pondere de 90% iar cea a descrierilor de 10%.

5.6 Inițializarea sistemului

Diagrama de secvență [5.3](#) oferă o explicație vizuală asupra transferului și schimbului de date dintre sistemul de recomandare și aplicația client.

Inițial, sistemul de recomandare necesită cunoașterea unui identificator a fiecărui utilizator din aplicația client. Aplicația client va fi nevoită în faza de inițializare a sistemului să transmită identificatorii împreună cu caracteristicile tuturor produselor.

Odată ce toate produsele au fost adăugate, aplicația client are posibilitatea de a semnala acest eveniment, deci, sistemul de recomandare poate codifica caracteristicile produselor folosind metoda one-hot encoding. Această tehnică produce un vector de lungimea numărului de caracteristici unice ale produselor. De exemplu, dacă până într-un moment nu a existat un produs cu culoarea verde, adăugarea unui astfel de produs va necesita recalcularea tuturor vectorilor întrucât cei noi vor avea cu un element în plus.

Următorul pas este ca aplicația client să transmită acțiunile utilizatorilor asupra produselor. Din momentul în care relațiile au fost adăugate, prin semnalarea acestui eveniment profilele tuturor utilizatorilor vor fi create. În urma calculării profilelor utilizatorilor, sistemul este pregătit să genereze recomandări.

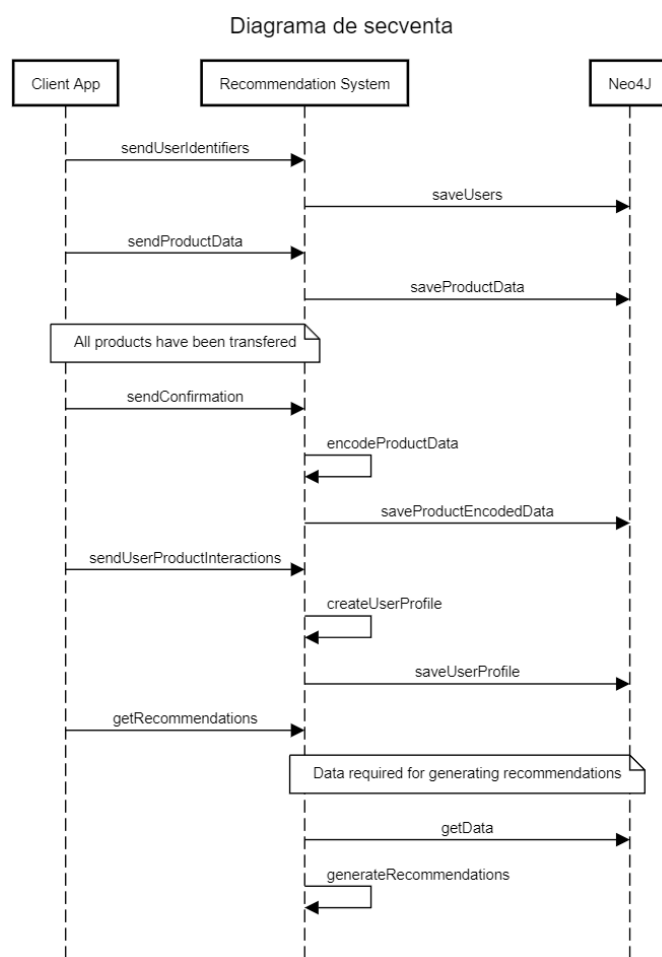


Figura 5.3: Diagramă de secvență a inițializării sistemului; Etapele de transfer și prelucrare a datelor

Capitolul 6

Testare și validare

În continuare se va prezenta validarea și corectitudinea sistemului de recomandare implementat prin teste unitare, folosirea platformei sonarqube și testarea interacțiunii unui utilizator cu sistemul printr-o aplicație demo.

6.1 Teste unitare

Testarea unitară servește ca o unealtă în procesul de validare a sistemului nostru de recomandare, oferind mai multe avantaje în asigurarea fiabilității și funcționalității acestuia. Izolând unitățile individuale de cod și supunându-le unor teste riguroase, putem identifica și rectifica erorile la începutul ciclului de dezvoltare.

De-a lungul fazei de implementare o multitudine de teste se concentrează pe validarea comportamentului funcțiilor, metodelor sau claselor individuale, examinând intrările, ieșirile și interacțiunile acestora în diferite scenarii. Aceste teste promovează modularitatea codului și facilitează depanarea mai ușoară prin localizarea problemelor în anumite unități de cod.

Primele teste implementate au avut rolul de verificare a componentelor **Controller**, atât prin preluarea datelor cât și prin returnarea corectă a răspunsurilor. Testele pentru controllerul responsabil de crearea și ștergerea entității "User" din sistem au fost implementate prin mimarea unor date despre un utilizator și transmiterea acestora spre punctul final respectiv într-un context de testare al aplicației. Aceste teste mimează comportamentul serviciilor folosite de către componentele **controllere** pentru a se concentra pe rolul acestora, acela de a coordona transferul datelor între **View** și componentele **Service**.

Restul componentelor de acest tip (**Relation**, **Data**, **Product Controller**) au fost implementate într-o manieră similară, făcând excepție cele responsabile de recomandări. Întrucât aceste componente necesită returnarea unor obiecte din baza de date serializate, testele s-au asigurat că obiectele returnate sunt adecvate transiterii spre aplicația client.

Testele pentru componentele **Service** au fost implementate prin mimarea modelelor

entităților, inițializate cu date de testare, pentru a verifica logica pe care componentele o execută. De asemenea, s-a verificat numărul de apeluri ale funcțiilor din interiorul acestor componente pentru a ne asigura că numărul apelurilor este unul așteptat.

A fost necesară testarea fiecărui scenariu a serviciului care generează recomandări deoarece consider că este cel mai important. Primul scenariu testat a fost cazul în care aplicația client cere recomandări pentru un utilizator neautentificat sau despre care sistemul nu deține informații. În acest caz serviciul trebuie să returneze trei obiecte serializate în ordinea popularității. Al doilea scenariu testat a fost cazul în care sistemul deține informații despre utilizator și aceste trebuie să returneze trei obiecte în ordinea relevanței.

6.2 SonarQube

SonarQube este o platformă open-source concepută pentru inspecția bazelor de cod. Această tehnologie oferă un set cuprinzător de instrumente pentru analiza statică a codului, revizuirea codului și managementul calității software-ului, facilitând detectarea bug-urilor și vulnerabilităților într-o gamă largă de limbaje de programare.¹

În raportul inițial **6.1** se poate observa că au fost detectate 0 buguri, 0 vulnerabilități de securitate, 3 componente care necesită atenție când vine vorba de securitate și 10 *code smells*. Termenul de *code smell* se referă la practicile mai puțin bune din cadrul codului, ce pot duce la probleme pe termen lung. Acestea nu sunt bug-uri, însă indică o porțiune unde calitatea codului poate fi îmbunătățită.

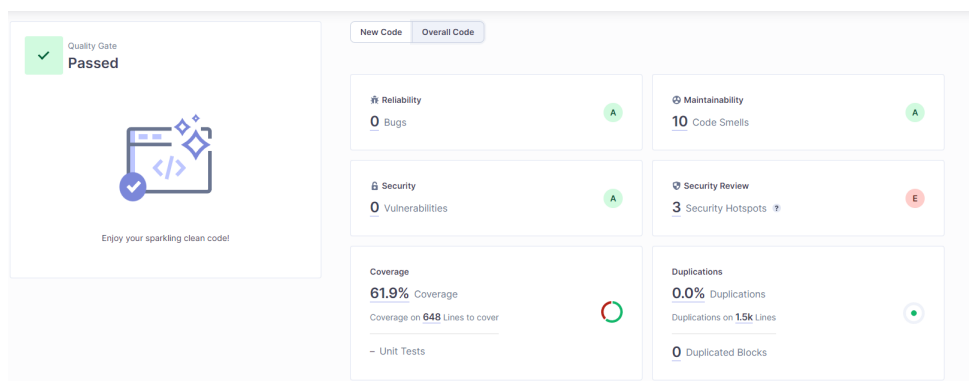


Figura 6.1: SonarQube raport inițial

Având în vedere că majoritatea problemelor de la secțiunea code smells au legătură cu nerespectarea convențiilor de denumire a funcțiilor și a claselor, acestea au fost rezolvate prin simpla redenumire a acestora. În tabelul următor **6.1** se prezintă problemele care nu au legătură cu convențiile de denumire.

¹<https://www.sonarsource.com/products/sonarqube/downloads/>

Code smell	Rezolvare
Funcția <code>init</code> (constructorul) a clasei <code>Embedding</code> nu realizează inițializarea modelului; modelul este inițializat implicit ca atribut al clasei	Inițializarea modelului se face explicit în funcția <code>init</code> , asigurând o inițializare clară și explicită
Verificare unei variabile dacă nu este egală cu obiectul <code>None</code> când nu există un altfel de scenariu	Eliminarea verificării

Tabelul 6.1: Code smells

Prima problemă din secțiunea Security Review indică că protecția Cross-Site Request Forgery (CSRF) este dezactivă. Ținând cont că proiectul nu va fi pus într-un mediu de producție, am ales să nu configurez protecția CSRF.

Majoritatea motoarelor de expresii regulate folosesc *backtracking* pentru a încerca toate căile de execuție posibile ale expresiei regulate, astfel există o vulnerabilitate la atacuri denial-of-service (DoS). Cu gândul la prevenirea acestei probleme am implementat un wrapper care oprește execuția funcției dacă o anumită limită de timp a trecut fără finalizarea prelucrării. A mai fost adăugată o verificare a lungimii conținutului html transmis de către client.

Ultima problemă semnalată de către platforma SonarQube a fost lipsa unei origini specifice în configurarea CORS. Configurând originea ca fiind adresa și portul aplicației client, eliminăm posibilitatea ca alte aplicații să folosească sistemul.

În cele din urmă incosistențele și toate eventualele vulnerabilități ale sistemului au fost corectate, acest lucru reflectându-se în raportul final [6.2](#).

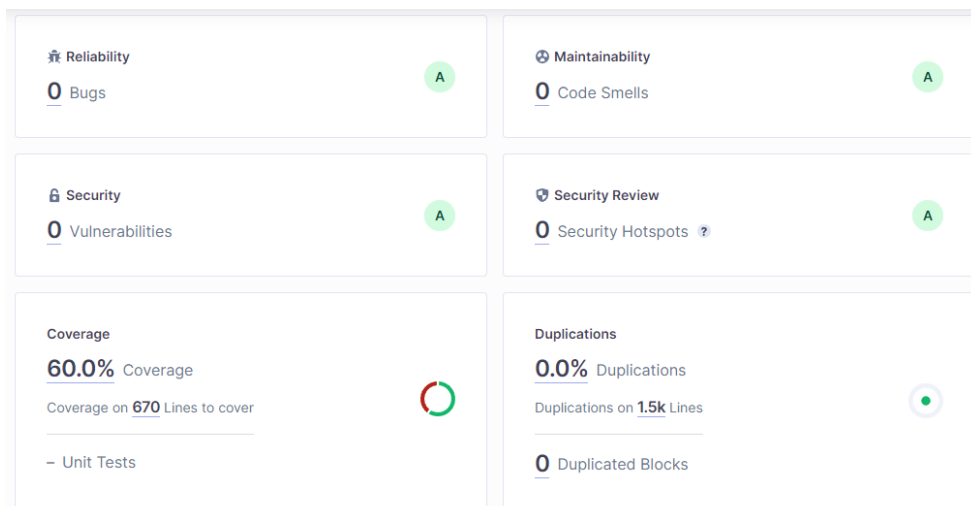


Figura 6.2: SonarQube raport final

6.3 Testare manuală

În această secțiune se va testa manual funcționarea sistemului de recomandare care a fost integrat cu o aplicație client dezvoltată folosind Spring Boot și React.

După înregistrarea unui utilizator în aplicația client sistemul de recomandare adaugă cu succes identificadorul aceluiași utilizator în baza sa de date. Sistemul răspunde cu codul 200 și cu un mesaj "Save successful".

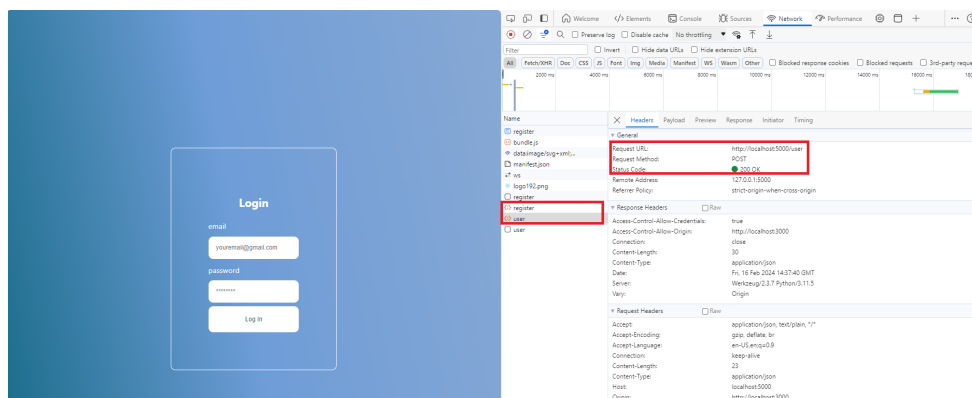


Figura 6.3: Înregistrarea unui utilizator în sistem

În imaginea [6.4](#) se poate observa că în cazul în care utilizatorul nu este autentificat, în urmă unei cereri, sistemul de recomandare răspunde cu trei recomandări bazate pe popularitatea acestora. În cazul în care un utilizator este autentificat dar nu a interacționat cu nici un produs sistemul va returna același răspuns.

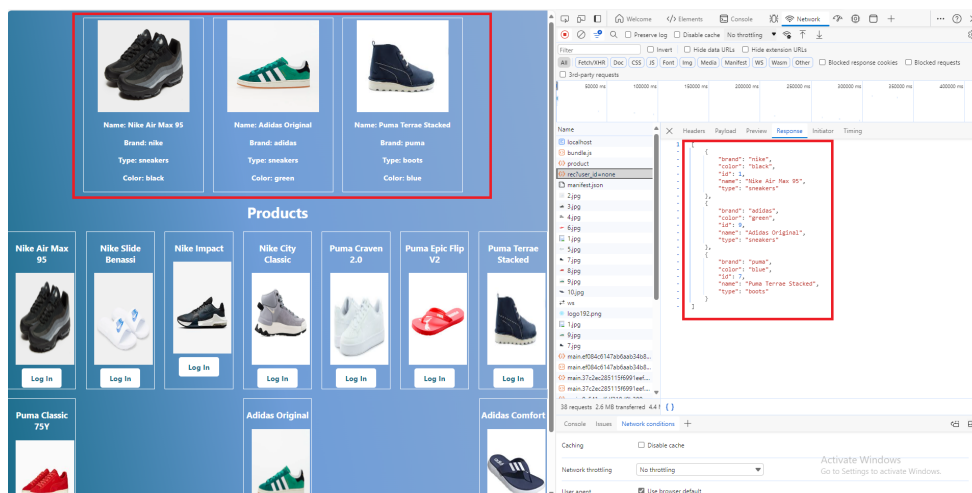


Figura 6.4: Recomandări bazate pe popularitate

În momentul în care utilizatorul da click pe pagina de detalii a unui produs, aplicația

client transmite această acțiune printr-o cerere care conține datele necesare. Răspunsul sistemului în acest caz provine din rezultatul algoritmului de filtrare bazată pe conținut. Se poate observa că prin exprimarea unui interes față de produsul vizualizat, sistemul de recomandare a returnat recomandări relevante pentru utilizator. Produsele recomandate sunt ordonate descrescător după relevanță.

Sistemul a returnat recomandări în aproximativ 2,5 secunde. Acest timp de răspuns este acceptabil încât nu afectează experiența utilizatorului într-un mod negativ.

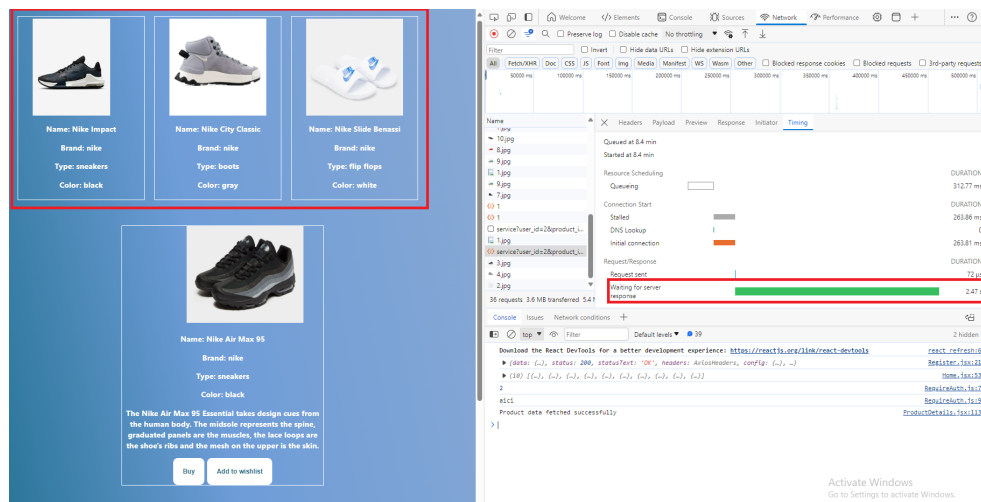


Figura 6.5: Interacțiunea "SEEN" cu un produs

A fost testată abilitatea sistemului de folosire a tehnicilor de NLP pentru a corecta ratingul oferit de un utilizator. Se poate observa în imaginea 6.6 că utilizatorul a oferit un rating de 2 deși comentariul este unul clar pozitiv. Greutatea relației salvată în baza de date este una de 0.7, aceasta indicând prezența unui rating ușor ridicat.

Întrucât modelul vader se bazează pe un dicționar, s-a testat abilitatea acestuia de a detecta sarcasmul sau ironia din propoziții. Au fost testate fraze cum ar fi "These shoes are like a rollercoaster ride for your feet, except the only thrill you'll experience is wondering when they'll fall apart." care au un subînțeles negativ. Modelul a oferit sentimentul neutru în majoritatea acestor cazuri. Dat fiind faptul că există o problemă la detectarea sarcasmului, în cazurile în care fraza are sentimentul neutru sentimentul comentariului nu va afecta ratingul în nici un fel.

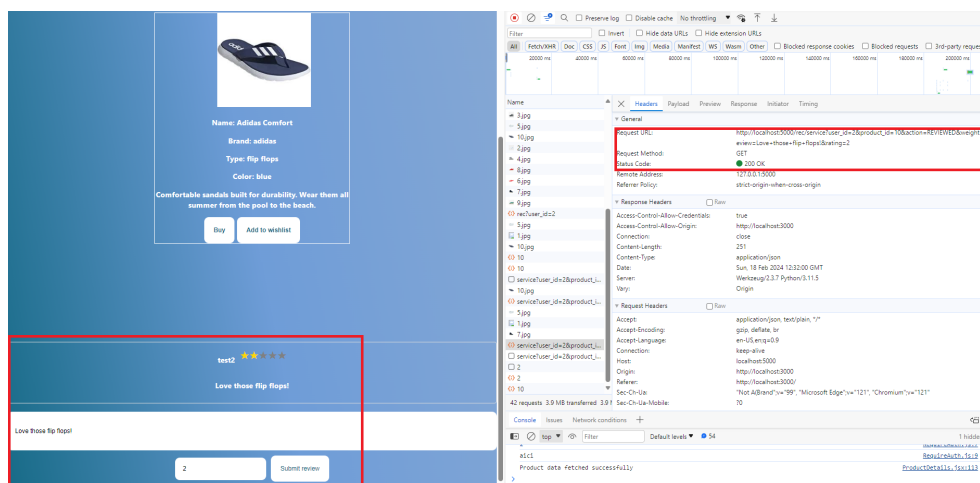


Figura 6.6: Transmiterea acțiunii REVIEWED

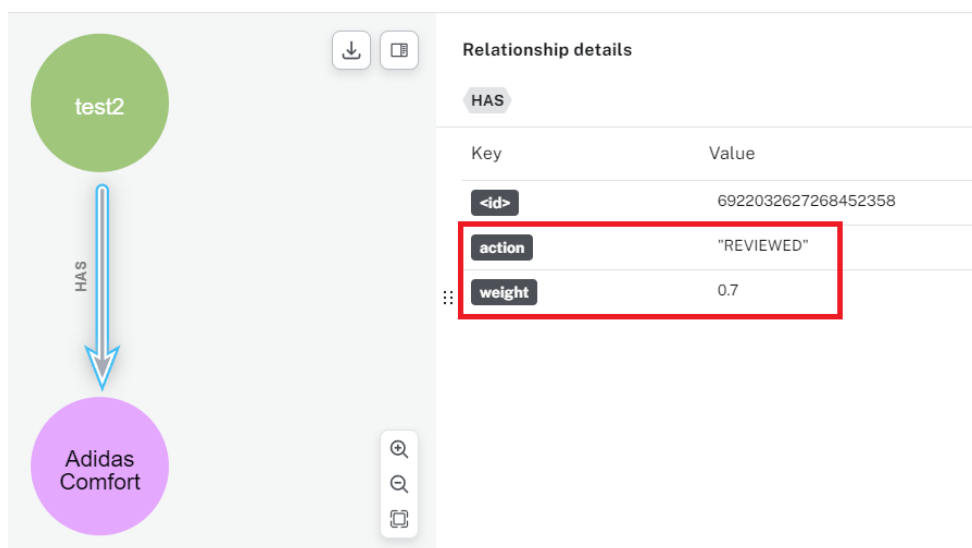


Figura 6.7: Greutatea relației

6.4 Evaluarea sistemului de recomandare prin interacțiunile cu utilizatorii

Pentru crearea unui sistem de recomandare eficient, este necesară analiza performanței într-un scenariu real unde interacțiunile utilizatorilor joacă un rol crucial. În acest subcapitol vom explora metodologia de validare interactivă, aceasta implicând activ utilizatorii.

Au fost aleși 10 participanți la procesul de validare, fiecare făcând parte dintre diferite grupuri de vârstă, interese și istoric. Această diversitate oferă sistemului interacțiuni

6.4. EVALUAREA SISTEMULUI DE RECOMANDARE PRIN INTERACȚIUNILE CU UTILIZATORII

cu utilizatori din diferite segmente, astfel oferind diferite perspective asupra performanței sistemului. Din rațiuni de protecție a anonimatului, vom expune doar identificatorii utilizatorilor.

Primul pas al acestui proces este ca utilizatorii să interacționeze cu sistemul când acesta nu deține nici un fel de informație despre preferințele acestora. Participanții vor oferi feedback despre recomandările generate aleator [6.2](#).

Identificator utilizator	Numărul de produse relevante
0	0/3
1	0/3
2	0/3
3	1/3
4	1/3
5	0/3
6	1/3
7	0/3
8	0/3
9	0/3

Tabelul 6.2: Relevanța recomandărilor generate aleator

Următorul pas este alegerea unui subset de 5 participanți care va interacționa cu produsele. La acest subset de participanți ne vom referi de acum încolo ca grupul "A", iar la restul participanților grupul "B". Grupul A va interacționa cu produsele aplicației, astfel, sistemul va strânge informații despre preferințele acestora. După interacționarea cu produsele, se va compara feedback-ul inițial oferit de către grupul A cu feedback-ul oferit recomandărilor în urma interacțiunilor.

În tabelul [6.3](#) putem observa recomandările generate de către sistem după ce fiecare utilizator din grupul A a avut în medie 3 interacțiuni (vizualizare, cumpărare etc.) cu produsele de pe platforma client. Numărul produselor relevante returnate tuturor utilizatorilor din grupului A înainte de interacțiunile cu sistemul a fost 1 din 15, ulterior interacțiunilor numărul a crescut la 7 din 15. Sistemul a returnat cu 600% mai multe produse relevante după ce acesta a folosit informațiile despre preferințele utilizatorilor.

De asemenea, am documentat și numărul interacțiunilor cu sistemul. Această metrică poate fi interpretată ca fiind abilitatea sistemului de a ajuta utilizatorii în căutarea și descoperirea produselor.

Identificator utilizator	Numărul de produse relevante	Numărul de interacțiuni cu sistemul
3	1/3	2
1	1/3	1
2	2/3	1
0	1/3	1
9	2/3	1

Tabelul 6.3: Relevanța recomandărilor oferite grupului A

În urma interacțiunilor efectuate de grupul A, sistemul va oferi recomandări bazate pe popularitate utilizatorilor din grupul B. Comparând feedback-ul oferit inițial cu cel oferit recomandărilor bazate pe popularitate putem valida această tehnică de generare în cazul în care nu exista informații despre utilizator.

În tabelul 6.4 putem observa relevanța recomandărilor bazate pe popularitate oferite grupului B. Numărul de produse relevante returnate aleator grupului B a fost de 2 din 15, ulterior interacționării grupului A cu platforma acest număr a crescut la 4 din 15. Această tehnică de generare a recomandărilor a returnat cu 100% mai multe produse relevante.

Identificator utilizator	Numărul de produse relevante
4	1/3
5	1/3
6	1/3
7	1/3
8	0/3

Tabelul 6.4: Relevanța recomandărilor oferite grupului B

Un impediment întâlnit în procesul de validare a fost numărul mic de produse oferit de platforma client. Întrucât platforma oferă 25 de produse, șansa ca multiple produse asemănătoare să existe este mai mică, prin urmare relevanța recomandărilor poate scădea. Deoarece sistemul nu returnează produse cu care utilizatorii au interacționat deja, faptul că aceștia interacționau de la început cu produsele relevante lor făceau ca relevanta recomandărilor să se deterioreze.

Ținând cont de rezultatele obținute, putem afirma că sistemul implementat are abilitatea de a se mula pe preferințele utilizatorilor astfel oferindu-le produse relevante.

Capitolul 7

Manual de instalare

Obiectivul urmărit de acest capitol este de a oferi utilizatorilor un manual cu instrucțiunile necesare pentru instalarea sistemului de recomandare. Deoarece sistemul a fost implementat local, este necesară deținerea codului sursă.

Astfel, pașii care trebuie urmați pentru instalare sunt următorii:

1. Clonarea codului sursă folosind comanda "git clone <https://github.com/BVancea25/API-for-RS/tree/actual>", rulată în directorul în care se dorește descărcarea.
2. Instalarea limbajului de programare python versiunea 3.11.5.
3. Adăugarea python la variabilele de mediu.
4. După navigarea în directorul aplicației rularea comenzii "pip install -r requirements.txt" pentru instalarea dependintelor în mediu.
5. Crearea unui cont pe platforma neo4j la url-ul "<https://neo4j.com/cloud/platform/aura-graph-database/?ref=nav-get-started-cta>".
6. Odată autentificați, crearea unei instanțe neo4j.

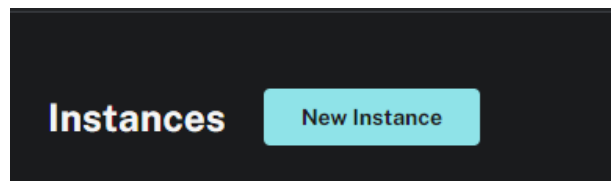


Figura 7.1: Crearea unei instanțe

7. Descărcarea de pe platformă a fișierului care conține următoarele date despre instanță:: NEO4J_URI, NEO4J_USERNAME si NEO4J_PASSWORD.
8. Navigarea la calea "./src/Config" în directorul proiectului.

9. Completarea datelor necesare în fișierul db.py folosind datele din fișierul descărcat anterior.
10. Folosirea comenzii "python index.py" pentru rularea proiectului pe portul 5000.
11. Folosirea endpointului "/user" 7.2 cu metoda "GET" pentru a transmite identicatorul unui utilizator spre sistem. Identificatorul de tip întreg va fi prezent în corpul cererii cu cheia "id", de asemenea, va fi prezentă o cheie "name" a cărei valori va fi un șir de caractere reprezentând numle utilizatorului. Ca și răspuns la cerere se returnează o pereche cheie valoare care conține un mesaj descriptiv.

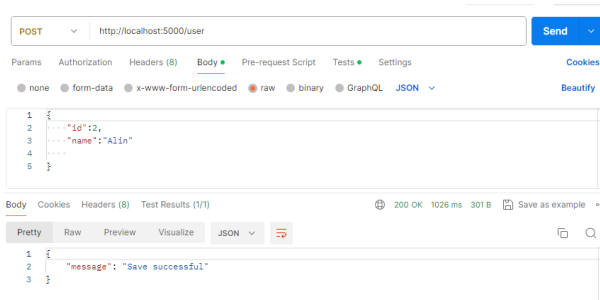


Figura 7.2: Crearea unui utilizator prin trimiterea unei cereri

12. Transmiterea datelor despre produse 7.3 la endpointul "/product" metoda "POST". În contextul cazului de utilizare pentru care sistemul a fost implementat, se vor transmite date despre pantofi în corpul cererii cu următoarele chei: "type", "description", "name", "brand" și "color".



Figura 7.3: Transmiterea datelor unui produs

13. Semnalarea sistemului 7.4 de recomandare printr-o cerere cu metoda "PUT" la endpointul "/data" indică că toate produsele au fost adăugate și că codificarea caracteristicilor poate începe.

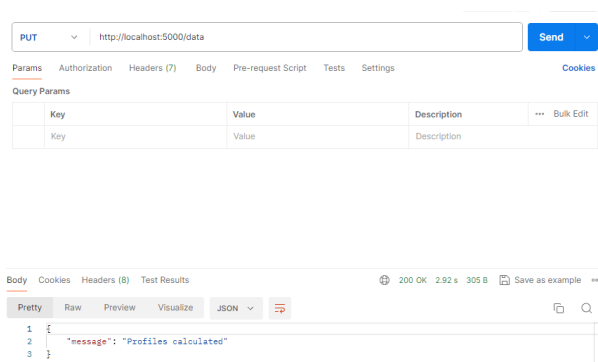


Figura 7.4: Cerere la endpointul "/data"

14. Folosirea endpointului **7.5** "/relation" metoda "POST" pentru adăugarea interacțiunilor utilizator-produs în sistem. Corpul cererii va conține următoarele chei: "user_id" (int), "product_id" (int), "action" (string) și "weight" (int). În cazul în care este o interacțiune de tip "REVIEWED" corpul cererii va conține de asemenea "review"(string) și "rating"(int).

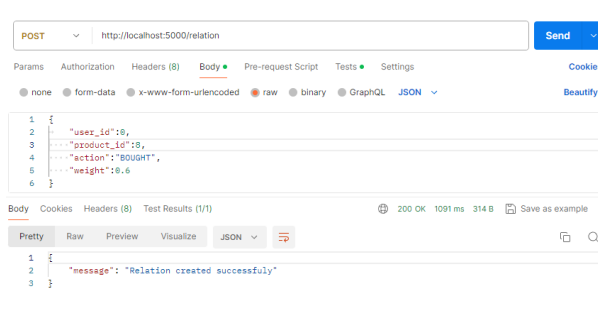


Figura 7.5: Adăugarea unei relații

15. Semnalarea sistemului de recomandare prin transmiterea unei cereri **7.6** la endpointul "/user" cu metodă "PUT" că calcularea profilelor utilizatorilor poate începe.

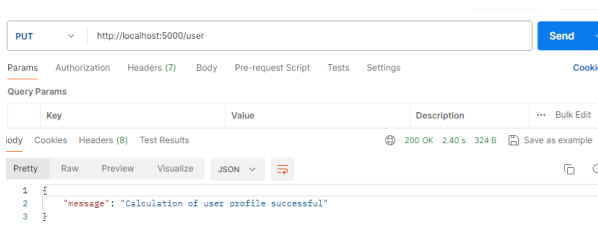


Figura 7.6: Calcularea profilelor utilizatorilor

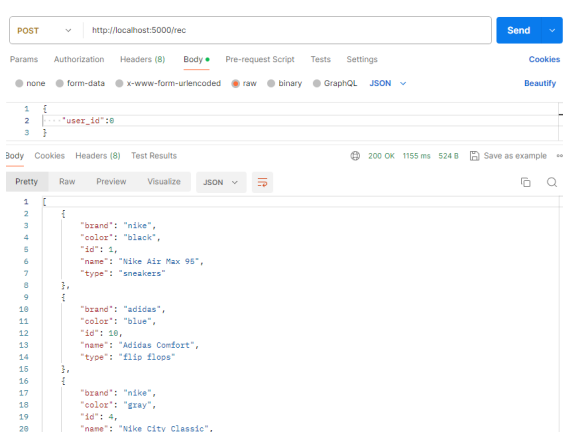


Figura 7.7: Obținerea de recomandări

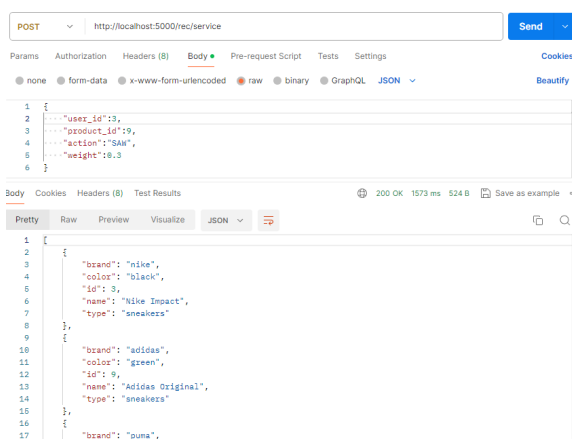


Figura 7.8: Obținerea de recomandări în urma unei acțiuni

16. Folosirea endpointului **7.7** ”/rec” metoda ”POST” pentru obținerea a trei produse relevante pentru un anumit utilizator. Id-ul utilizatorului va fi prezent în corpul cererii cu cheia ”user_id”(int). Răspunsul dat de endpoint este sub forma unei liste de produse. Fiecare dintre obiectele din lista conține caracteristicile împreună cu identificatorul fiecăruia. În cazul în care se doresc recomandări pentru un utilizator neautentificat se poate transmite prin același mod o cere, însă valoarea cheii ”user_id” (sting) va avea valoarea ”none”.
17. În cazul în care utilizatorul efectuează o acțiune asupra unui produs și se doresc recomandări actualizate cu acea acțiune **7.8**, se va folosi endpointul ”/rec/service” metoda ”POST”. Datele corespunzătoare se vor afla în corpul cererii cu cheile: ”product_id”(int), ”user_id”(int) împreună cu datele menționate la pasul 15. Recomandările din răspuns sunt structurate într-o listă de obiecte. Fiecare dintre ele conține caracteristicile și identificatorul propriu.

Capitolul 8

Concluzii

Sistemul propus oferă aplicațiilor client recomandări relevante pentru utilizatorii lor folosind datele produselor și interacțiunile utilizator-produs.

Folosind la bază algoritmul de filtrare bazată pe conținut [4.3](#), în urma testării și validări se poate constata că sistemul reușește să atingă obiectivele propuse [2.1](#).

Acest algoritm are câteva avantaje față de alți algoritmi care se bazează pe tehnici de învățare automată sau pe filtrarea colaborativă [4.1](#). Problema de cold-start când vine vorba de produse noi intrate în sistem este evitată, întrucât sistemul face recomandări pe baza caracteristicilor acestora. Algoritmul oferă transparență în procesul de generare a recomandărilor, astfel ducând la o interacțiune tehnologie-utilizator lipsită de fricțiune.

Descrierile produselor au fost cu succes introduse în procesul de generare a recomandărilor, astfel folosind eventualele informații contextuale prezente în descrieri cu scopul îmbunătățirii relevanței produselor recomandate. Prin folosirea modelului SBERT [4.2.4](#), propozițiile care compun descrierile produselor au fost transformate în vectori pentru prelucrări ulterioare.

Datorită arhitecturii Model-View-Controller-Service și testelor efectuate [6](#), a fost implementată o bază de cod modulară și corect funcțională. Pe deasupra, interfața REST oferă aplicațiilor client o integrare ușoară cu sistemul implementat.

Totuși, algoritmul de filtrare bazată pe conținut are câteva limitări. Cea mai notabilă este problema cold-start când vine vorba de utilizatori noi în sistem. Deoarece acești utilizatori nu au interacționat cu nici un produs, sistemul nu poate genera nici un fel de recomandare. O soluție temporară a fost implementată, aceea de a recomanda acestor utilizatori produsele în funcție de popularitatea acestora. Această soluție sa dovedit a fi performantă, oferind un număr de 2 ori mai mare de produse relevante utilizatorilor.

Sistemul reușește să se muleze pe preferințele utilizatorilor [6.4](#), astfel oferind de 6 ori mai multe produse relevante fata de oferirea unor produse aleatorii.

8.1 Dezvoltări ulterioare

O îmbunătățire a sistemului implementat ar fi adăugarea unei tehnici de filtrare colaborativă bazată pe utilizatori [4.1], la adăugarea acestei tehnici baza de date graf va fi un pilon esențial, întrucât se vor parcurge relații multiple între utilizatori. Introducerea unor date contextuale în procesul generării de recomandări, astfel creând un sistem hybrid care ar putea crește relevanța rezultatelor. Deși acest sistem îmbunătățit ar aduce recomandări mai relevante, un dezavantaj ar putea fi transparența procesului de generare a acestora.

Modul în care sunt agregați vectorii de valori încorporate ale descrierilor s-ar putea să necesite atenție. Prin calcularea mediei valorilor încorporate s-ar putea pierde înțelegerea descrierilor.

Întrucât caracteristicile produselor nu sunt necesare după calcularea profilelor, o optimizare a spațiului de stocare ar fi introducerea unei baze de date redis pentru stocarea caracteristicilor până în momentul calculării profilelor.

Folosirea unui algoritm genetic ar putea fi benefică pentru optimizarea greutăților folosite de sistem. Ca și funcție de fitness s-ar putea folosi o metrică de performanță specifică sistemelor care returnează o lista de predicții cum ar fi NDCG [14].

Prin urmare, s-a implementat cu succes un sistem de recomandare folosind tehnicile menționate, acesta fiind capabil de a îmbunătăți experiența utilizatorilor prin recomandările relevante și prin abilitatea de a ajuta în procesul de descoperire și căutare a produselor.

Bibliografie

- [1] D. Roy and M. Dutta, “A systematic review and research perspective on recommender systems,” *Journal of Big Data*, vol. 9, no. 1, p. 59, 2022.
- [2] G. Castellano, A. M. Fanelli, and M. A. Torsello, “Newer: A system for neuro-fuzzy web recommendation,” *Applied Soft Computing*, vol. 11, no. 1, pp. 793–806, 2011.
- [3] P. Ochi, S. Rao, L. Takayama, and C. Nass, “Predictors of user perceptions of web recommender systems: How the basis for generating experience and search product recommendations affects user responses,” *International Journal of Human-Computer Studies*, vol. 68, no. 8, pp. 472–482, 2010.
- [4] N. Kurmashov, K. Latuta, and A. Nussipbekov, “Online book recommendation system,” in *2015 Twelve International Conference on Electronics Computer and Computation (ICECCO)*. IEEE, 2015, pp. 1–4.
- [5] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou, “Mobile recommender systems in tourism,” *Journal of network and computer applications*, vol. 39, pp. 319–333, 2014.
- [6] H. Ko, S. Lee, Y. Park, and A. Choi, “A survey of recommendation systems: recommendation models, techniques, and application fields,” *Electronics*, vol. 11, no. 1, p. 141, 2022.
- [7] R. Zhou, S. Khemmarat, and L. Gao, “The impact of youtube recommendation system on video views,” pp. 404–410, 2010.
- [8] M. B. Dias, D. Locher, M. Li, W. El-Deredy, and P. J. Lisboa, “The value of personalised recommender systems to e-business: a case study,” pp. 291–294, 2008.
- [9] R. Sinha and K. Swearingen, “The role of transparency in recommender systems,” in *CHI’02 extended abstracts on Human factors in computing systems*, 2002, pp. 830–831.
- [10] D. M. Fleder and K. Hosanagar, “Recommender systems and their impact on sales diversity,” pp. 192–199, 2007.

- [11] E. E. Erdem and G. K. Orman, “On the role of user segmentation in recommender systems’ performances,” *Procedia Computer Science*, vol. 225, pp. 2333–2342, 2023.
- [12] P. Shoval, V. Maidel, and B. Shapira, “An ontology-content-based filtering method,” 2008.
- [13] J. Song, C. Chang, F. Sun, X. Song, and P. Jiang, “Ngat4rec: Neighbor-aware graph attention network for recommendation,” *arXiv preprint arXiv:2010.12256*, 2020.
- [14] H. Valizadegan, R. Jin, R. Zhang, and J. Mao, “Learning to rank by optimizing ndcg measure,” *Advances in neural information processing systems*, vol. 22, 2009.
- [15] G. Adomavicius and A. Tuzhilin, “Context-aware recommender systems,” pp. 217–253, 2010.
- [16] P. Buono, M. F. Costabile, S. Guida, and A. Piccinno, “Integrating user data and collaborative filtering in a web recommendation system,” pp. 315–321, 2002.
- [17] M. Kurovski and F. Wilhelm, “On the effectiveness of low-rank approximations for collaborative filtering compared to neural networks,” *arXiv preprint arXiv:1905.12967*, 2019.
- [18] C. A. Gomez-Uribe and N. Hunt, “The netflix recommender system: Algorithms, business value, and innovation,” *ACM Transactions on Management Information Systems (TMIS)*, vol. 6, no. 4, pp. 1–19, 2015.
- [19] J. Gope and S. K. Jain, “A survey on solving cold start problem in recommender systems,” pp. 133–138, 2017.
- [20] C. Ono, M. Kurokawa, Y. Motomura, and H. Asoh, “A context-aware movie preference model using a bayesian network for recommendation and promotion,” pp. 247–257, 2007.
- [21] S. Batra, V. Sharma, Y. Sun, X. Wang, and Y. Wang, “Steam recommendation system,” *arXiv preprint arXiv:2305.04890*, 2023.
- [22] S. Gong, “A collaborative filtering recommendation algorithm based on user clustering and item clustering,” *J. Softw.*, vol. 5, no. 7, pp. 745–752, 2010.
- [23] Y. Zheng, B. Mobasher, and R. Burke, “Cslim: Contextual slim recommendation algorithms,” pp. 301–304, 2014.
- [24] A. Tegene, Q. Liu, Y. Gan, T. Dai, H. Leka, and M. Ayenew, “Deep learning and embedding based latent factor model for collaborative recommender systems,” *Applied Sciences*, vol. 13, no. 2, p. 726, 2023.

- [25] Y. Yang, H.-J. Jang, and B. Kim, "A hybrid recommender system for sequential recommendation: combining similarity models with markov chains," *IEEE Access*, vol. 8, pp. 190 136–190 146, 2020.
- [26] W. Shafqat and Y.-C. Byun, "A context-aware location recommendation system for tourists using hierarchical lstm model," *Sustainability*, vol. 12, no. 10, p. 4107, 2020.
- [27] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," pp. 785–794, 2016.
- [28] O. Melamud, J. Goldberger, and I. Dagan, "context2vec: Learning generic context embedding with bidirectional lstm," pp. 51–61, 2016.
- [29] R. Dey and F. M. Salem, "Gate-variants of gated recurrent unit (gru) neural networks," pp. 1597–1600, 2017.
- [30] F. ZhuanSun, J. Chen, W. Chen, Y. Sun *et al.*, "Analysis of precision service of agricultural product e-commerce based on multimodal collaborative filtering algorithm," *Mathematical Problems in Engineering*, vol. 2022, 2022.
- [31] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," pp. 974–983, 2018.
- [32] B. Jiang and D. Lin, "Graph laplacian regularized graph convolutional networks for semi-supervised learning," *arXiv preprint arXiv:1809.09839*, 2018.
- [33] R. Burioni and D. Cassi, "Random walks on graphs: ideas, techniques and results," *Journal of Physics A: Mathematical and General*, vol. 38, no. 8, p. R45, 2005.
- [34] S. S. Salam, P. Dhanalakshmi, N. L. Anantha, and A. Sandeep, "Movie recommendation system using deep learning techniques," *Journal of Pharmaceutical Negative Results*, vol. 13, no. 4, pp. 1047–1051, 2022.
- [35] R. Burke, A. Felfernig, and M. H. Göker, "Recommender systems: An overview," *Ai Magazine*, vol. 32, no. 3, pp. 13–18, 2011.
- [36] R. Van Meteren and M. Van Someren, "Using content-based filtering for recommendation," vol. 30, pp. 47–56, 2000.
- [37] E. Çano and M. Morisio, "Hybrid recommender systems: A systematic literature review," *Intelligent data analysis*, vol. 21, no. 6, pp. 1487–1524, 2017.
- [38] F. Long, "Improved personalized recommendation algorithm based on context-aware in mobile computing environment," *Wireless Communications and Mobile Computing*, vol. 2020, pp. 1–10, 2020.

- [39] T. Winograd, “Procedures as a representation for data in a computer program for understanding natural language,” 1971.
- [40] G. Nagy, S. Seth, and M. Viswanathan, “A prototype document image analysis system for technical journals,” *Computer*, vol. 25, no. 7, pp. 10–22, 1992.
- [41] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [42] C. Hutto and E. Gilbert, “Vader: A parsimonious rule-based model for sentiment analysis of social media text,” in *Proceedings of the international AAAI conference on web and social media*, vol. 8, no. 1, 2014, pp. 216–225.
- [43] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” *arXiv preprint arXiv:1908.10084*, 2019.
- [44] E. E. Ian Robinson, Jim Webber, *Graph Databases New Opportunities for Connected Data*. O’Reilly Media, 2015.